

*Corso di Laurea Triennale in*  
**Ingegneria Elettronica e Informatica**

**Tesi di Laurea in Ingegneria del Software**

Progettazione e Sviluppo di un'Architettura  
a Microservizi per la Gestione di un Hub di  
Ricarica per Veicoli Elettrici

Candidato/a                      Relatore  
*Raimondo Panico*    *Prof. Antonio Esposito*  
Matr. A13002793

**A. A. 2025/2026**

*Alla mia famiglia, per i sacrifici silenziosi  
che mi hanno condotto a questo traguardo.*

*A Francesca, mia dolce metà  
e inesauribile fonte di energia.*

# Sommario

La crescente diffusione della mobilità elettrica impone infrastrutture software affidabili, scalabili e resilienti. I sistemi monolitici tradizionali, per quanto rapidi da realizzare in fase prototipale, rivelano i propri limiti architetturali non appena si rende necessario isolare un guasto o gestire un picco di carico su specifiche funzionalità.

Per superare queste limitazioni, questa tesi descrive la progettazione e lo sviluppo di un backend per hub di ricarica incentrato sul modello a microservizi. Realizzato in Java con Spring Boot, il sistema è stato organizzato in due domini logicamente separati: il Servizio Utenti, dedicato alla gestione delle anagrafiche e delle autorizzazioni, e il Servizio Colonnine, preposto al tracciamento dell'infrastruttura fisica. In accordo con il principio del database-per-service, ciascun modulo opera su un proprio livello di persistenza fisicamente isolato, interfacciandosi con gli altri moduli unicamente tramite API REST.

La gestione della concorrenza ha rappresentato uno degli scogli progettuali più complessi. Per scongiurare la doppia assegnazione di una colonnina appena liberata a causa di richieste simultanee, si è optato per un meccanismo transazionale di lock ottimistico. Questa scelta è risultata particolarmente efficace: garantisce l'integrità dei dati relazionali senza introdurre i rallentamenti tipici dei blocchi pessimistici, che avrebbero penalizzato l'intero sistema.

L'infrastruttura è stata infine validata tramite test a tre livelli: unità, integrazione con il database ed End-to-End. Gli esiti ottenuti dimostrano che il sistema orchestrato gestisce in modo corretto i flussi operativi, risolve in modo affidabile i conflitti di concorrenza e costituisce una base solida per futuri sviluppi orientati al cloud.

# Indice

|                                                                                         |           |
|-----------------------------------------------------------------------------------------|-----------|
| <b>Sommario</b>                                                                         | <b>2</b>  |
| <b>1 Introduzione</b>                                                                   | <b>7</b>  |
| 1.1 Il Dominio Applicativo: L'Infrastruttura di Ricarica e il Protocollo OCPP . . . . . | 7         |
| 1.2 Evoluzione architetturale: Stato dell'Arte . . . . .                                | 8         |
| <b>2 Architettura e Progettazione del Sistema Distribuito</b>                           | <b>10</b> |
| 2.1 Contesto e Obiettivi del Progetto . . . . .                                         | 10        |
| 2.2 Specifica dei Requisiti del Sistema . . . . .                                       | 12        |
| 2.2.1 Requisiti Funzionali . . . . .                                                    | 12        |
| 2.2.2 Requisiti Non Funzionali . . . . .                                                | 12        |
| 2.3 Pattern di comunicazione e coordinamento distribuito . . . . .                      | 13        |
| 2.3.1 Gestione dei Fallimenti Distribuiti: Il Pattern Saga . . . . .                    | 16        |
| 2.3.2 Il Teorema CAP e i trade-off architetturali . . . . .                             | 17        |
| 2.4 Controllo della concorrenza e integrità transazionale . . . . .                     | 17        |
| 2.5 Modellazione della persistenza e configurazione degli strati . . . . .              | 19        |
| 2.6 Analisi dei Rischi e Sicurezza delle API . . . . .                                  | 21        |
| 2.7 Conclusioni del Capitolo . . . . .                                                  | 22        |
| <b>3 Tecnologie e Strumenti Utilizzati</b>                                              | <b>23</b> |
| 3.1 Ecosistema Java e Spring Boot . . . . .                                             | 23        |
| 3.2 Persistenza dei Dati: Spring Data JPA e H2 Database . . . . .                       | 24        |
| 3.2.1 Dietro le quinte di Hibernate: la traduzione SQL di @Version . . . . .            | 24        |
| 3.2.2 Scelte infrastrutturali: H2 Database vs Testcontainers . . . . .                  | 25        |
| 3.3 Gestione delle Dipendenze e Build: Apache Maven . . . . .                           | 26        |
| <b>4 Implementazione dei Microservizi con Spring Boot</b>                               | <b>27</b> |
| 4.1 Modellazione e Sviluppo del Servizio Utenti . . . . .                               | 27        |
| 4.1.1 Esposizione delle API e Orchestrazione REST . . . . .                             | 29        |

|          |                                                                                    |           |
|----------|------------------------------------------------------------------------------------|-----------|
| 4.1.2    | Data Transfer Object (DTO) e Validazione dell'Input . . .                          | 32        |
| 4.1.3    | Documentazione Contratti REST: OpenAPI e Swagger . .                               | 33        |
| 4.2      | Sviluppo del Servizio Colonnine e Optimistic Locking . . . . .                     | 34        |
| 4.2.1    | Disaccoppiamento della Logica di Business: Il Service Layer                        | 36        |
| 4.2.2    | Logica di Business e Gestione del Conflitto HTTP 409 . .                           | 38        |
| 4.2.3    | Refactoring Architetturale: Gestione Centralizzata delle<br>Eccezioni . . . . .    | 39        |
| 4.3      | Popolamento del Database in Memoria (Data Loader) . . . . .                        | 42        |
| 4.4      | Limiti Noti e Sviluppi Futuri . . . . .                                            | 44        |
| 4.5      | Conclusioni del Capitolo . . . . .                                                 | 44        |
| <b>5</b> | <b>Collaudo e Validazione del Sistema</b>                                          | <b>45</b> |
| 5.1      | Strategia e obiettivi del collaudo . . . . .                                       | 45        |
| 5.2      | Unit Test: Validazione della Logica di Business . . . . .                          | 46        |
| 5.3      | Integration Test: Concorrenza e Optimistic Locking . . . . .                       | 47        |
| 5.4      | Collaudo End-to-End (E2E) Cross-Servizio . . . . .                                 | 48        |
| 5.5      | Discussione dei risultati e limiti del collaudo . . . . .                          | 51        |
| 5.6      | Conclusioni del capitolo . . . . .                                                 | 51        |
| <b>6</b> | <b>Conclusioni e Sviluppi Futuri</b>                                               | <b>52</b> |
| 6.1      | Conclusioni . . . . .                                                              | 52        |
| 6.2      | Sviluppi Futuri . . . . .                                                          | 52        |
| 6.2.1    | Containerizzazione e Infrastruttura DevOps: Docker e Doc-<br>ker Compose . . . . . | 53        |
|          | <b>Appendice: Configurazione Maven (pom.xml)</b>                                   | <b>56</b> |
|          | <b>Bibliografia e Sitografia</b>                                                   | <b>58</b> |
|          | <b>Ringraziamenti</b>                                                              | <b>60</b> |

# Elenco delle figure

|     |                                                                                                                                       |    |
|-----|---------------------------------------------------------------------------------------------------------------------------------------|----|
| 2.1 | Diagramma dei Casi d’Uso: interazione del Guidatore EV con i confini del sistema. . . . .                                             | 11 |
| 2.2 | Diagramma di Sequenza: orchestrazione della chiamata REST per l’avvio della sessione di ricarica. . . . .                             | 14 |
| 2.3 | Diagramma delle Classi: partizionamento del dominio e isolamento dei dati tra i microservizi. . . . .                                 | 19 |
| 2.4 | Schema E-R: Separazione fisica delle tabelle relazionali sui due database isolati. . . . .                                            | 21 |
| 5.1 | Risposta HTTP 200 OK generata in fase di collaudo End-to-End su Postman. . . . .                                                      | 49 |
| 5.2 | Gestione del conflitto: risposta HTTP 409 ottenuta al tentativo di prenotare una colonnina già occupata. . . . .                      | 50 |
| 6.1 | Diagramma di Deployment Concettuale: architettura a microservizi containerizzata con API Gateway e orchestrazione Kubernetes. . . . . | 53 |

# Elenco dei Codici

|      |                                                                                       |    |
|------|---------------------------------------------------------------------------------------|----|
| 3.1  | Traduzione SQL generata da Hibernate per il Lock Ottimistico . .                      | 24 |
| 3.2  | Configurazione application.properties per il database H2 . . . . .                    | 25 |
| 4.1  | Classe Entity Utente.java . . . . .                                                   | 27 |
| 4.2  | Interfaccia UtenteRepository.java . . . . .                                           | 28 |
| 4.3  | Classe UtenteController.java . . . . .                                                | 30 |
| 4.4  | Implementazione della classe RicaricaRequestDTO con validazione                       | 32 |
| 4.5  | Aggiornamento della firma del Controller con @Valid . . . . .                         | 33 |
| 4.6  | Annotazioni OpenAPI per la documentazione dell'endpoint . . . .                       | 34 |
| 4.7  | Classe Entity Colonnina.java . . . . .                                                | 34 |
| 4.8  | Interfaccia ColonninaRepository.java . . . . .                                        | 36 |
| 4.9  | Implementazione del Service Layer per la gestione transazionale .                     | 36 |
| 4.10 | Classe ColonninaController.java . . . . .                                             | 38 |
| 4.11 | Struttura del DTO standardizzato per gli errori . . . . .                             | 40 |
| 4.12 | Classe GlobalExceptionHandler per l'intercettazione globale degli<br>errori . . . . . | 40 |
| 4.13 | Classe DataLoader.java (Servizio Utenti) . . . . .                                    | 42 |
| 5.1  | Test di unità per la validazione dell'abbonamento tramite Mockito                     | 46 |
| 5.2  | Test di integrazione per la concorrenza e la risoluzione dei conflitti                | 47 |
| 6.1  | Esempio di Dockerfile Multi-stage per il microservizio . . . . .                      | 54 |
| 6.2  | Definizione del docker-compose.yml per l'intero cluster . . . . .                     | 54 |
| 6.3  | File pom.xml integrale del Servizio Colonnine . . . . .                               | 56 |

# Capitolo 1

## Introduzione

Negli ultimi anni, lo sviluppo di applicazioni enterprise si è progressivamente allontanato dal modello monolitico per abbracciare l'architettura a microservizi. Questo paradigma, che scompone il software in servizi indipendenti, piccoli e fortemente coesi, offre notevoli benefici in termini di scalabilità e resilienza, adattandosi perfettamente alle moderne infrastrutture cloud-native.

Il seguente lavoro di tesi si inserisce in questo scenario, con l'obiettivo di realizzare il backend per un sistema di gestione della ricarica di veicoli elettrici. Il dominio della mobilità elettrica richiede intrinsecamente sistemi reattivi, in grado di gestire accessi concorrenti e garantire una consistenza rigorosa dei dati. Per rispondere a queste esigenze, ho strutturato il progetto separando le responsabilità in due moduli indipendenti: un Servizio Utenti, dedicato alla gestione anagrafica e alle abilitazioni, e un Servizio Colonnine, incaricato di tracciare lo stato fisico e le prenotazioni delle infrastrutture di ricarica.

### 1.1 Il Dominio Applicativo: L'Infrastruttura di Ricarica e il Protocollo OCPP

Il dominio della mobilità elettrica (e-mobility) introduce sfide architetturali peculiari. Un hub di ricarica non è un semplice erogatore di energia, ma un vero ecosistema IoT (Internet of Things), dove hardware distribuito sul territorio deve comunicare costantemente con un sistema di gestione centrale, noto come CSMS (Charging Station Management System).

Le infrastrutture fisiche si distinguono principalmente in stazioni a corrente alternata (AC), per ricariche lente, e stazioni a corrente continua (DC) Fast e Ultrafast, essenziali per i nodi autostradali. Per attivare, monitorare e blocca-

re da remoto queste risorse, a prescindere dalla loro tipologia, è necessario un protocollo di comunicazione standardizzato..

Nell'industria odierna, lo standard *de facto* per questa comunicazione è l'OCPP (*Open Charge Point Protocol*). Si tratta di un protocollo applicativo (spesso implementato su WebSockets) che permette alla colonnina di inviare al backend messaggi critici, come *BootNotification* (all'accensione), *Heartbeat* (per segnalare di essere online) e *StatusNotification* (per comunicare se è disponibile, in uso o in errore).

Per definire il perimetro di questo lavoro, ho scelto di concentrarmi deliberatamente sugli strati alti del backend, tralasciando l'integrazione hardware specifica del protocollo OCPP (che in produzione richiederebbe un gateway dedicato). L'obiettivo centrale è infatti la modellazione dell'orchestrazione software e la gestione delle criticità transazionali. Per questo motivo, ho astratto le stazioni fisiche modellandone lo stato all'interno del database del Servizio Colonnine; questa scelta mi ha permesso di focalizzare lo sviluppo sulla tenuta del sistema di fronte a chiamate API concorrenti, che rappresenta la vera sfida ingegneristica del progetto.

Per lo sviluppo ho adottato Spring Boot, che mi ha permesso di realizzare API RESTful performanti in Java. La sfida progettuale più delicata è stata, tuttavia, la gestione della concorrenza: in un sistema distribuito, molti utenti possono tentare di prenotare la stessa risorsa. Ho scelto di implementare un meccanismo di Lock Ottimistico, che ho preferito ai lock pessimistici, perché mi garantiva l'integrità del database senza penalizzare il throughput complessivo del sistema.

## 1.2 Evoluzione architetturale: Stato dell'Arte

Prima della definitiva affermazione del modello a microservizi proposto da Fowler e Lewis (2014), la progettazione enterprise si è evoluta per gradi. Lo schema iniziale era il Monolite, dove interfaccia utente, logica di business e persistenza dei dati venivano compattati in un unico processo. Esisteva anche una variante più raffinata, il Monolite Modulare, che suddivideva il codice in pacchetti logici, ma manteneva inalterati il deployment unitario e lo strato di persistenza. Sebbene questo approccio semplifichi il tracciamento delle transazioni e azzeri le latenze di rete, esso presenta un limite strutturale: un picco di traffico su una funzionalità specifica obbliga a replicare l'intera applicazione.

Per superare questo limite, si è tentato con la Service-Oriented Architecture (SOA). La SOA divideva l'infrastruttura in servizi, ma li obbligava a comunicare

attraverso un Enterprise Service Bus (ESB) pesante e centralizzato. L'architettura a microservizi adottata in questo lavoro si pone come un'evoluzione della SOA, alleggerendola ed estremizzandola: l'ESB viene rimpiazzato da protocolli di rete standard (HTTP/REST) e si impone il database-per-service, garantendo così un reale disaccoppiamento tecnologico.

L'elaborato si articola nei seguenti capitoli:

- **Capitolo 2:** Analizza i principi teorici delle architetture a microservizi, con un focus sulla progettazione distribuita, il Teorema CAP e la modellazione del dominio.
- **Capitolo 3:** Delinea lo stack tecnologico adottato (Spring Boot, H2, Hibernate, Maven), giustificando le scelte di sviluppo e di persistenza.
- **Capitolo 4:** Entra nel vivo del progetto illustrando l'implementazione pratica in Java, la gestione del Lock Ottimistico e la documentazione dei contratti REST.
- **Capitolo 5:** Presenta la strategia di validazione del sistema, dai test di unità ai collaudi End-to-End (E2E) eseguiti con Postman.
- **Capitolo 6:** Raccoglie i risultati ottenuti, evidenzia i limiti dell'implementazione locale e suggerisce gli sviluppi futuri, in particolare la containerizzazione e l'evoluzione cloud.

## Capitolo 2

# Architettura e Progettazione del Sistema Distribuito

### 2.1 Contesto e Obiettivi del Progetto

L'affermarsi della mobilità elettrica ha imposto alle infrastrutture software una rapida evoluzione. Oggi questi sistemi devono essere estremamente affidabili e in grado di scalare orizzontalmente per sostenere il carico di lavoro. Tuttavia, il classico approccio monolitico, che compatta logica e database in un unico blocco, mostra spesso dei limiti in questi scenari: un semplice errore in un modulo secondario può bloccare l'intero servizio, e anche la più piccola modifica costringe a un rilascio completo dell'applicativo.

Per superare queste criticità strutturali, il mio lavoro di tesi mira a progettare e sviluppare una piattaforma per hub di ricarica interamente basata sul paradigma dei microservizi. In particolare, il progetto si prefigge tre obiettivi specifici:

- **Scomposizione del dominio:** separare nettamente le logiche di anagrafica e autorizzazione utente dal monitoraggio fisico delle colonnine.
- **Isolamento e resilienza:** implementare il pattern *Database-per-Service* per fare in modo che il guasto o il sovraccarico di un singolo modulo non comprometta l'intero sistema.
- **Gestione sicura della concorrenza:** prevenire la doppia assegnazione di una stessa colonnina a fronte di richieste simultanee, tutelando l'integrità dei dati.

L'idea di fondo è stata, quindi, quella di suddividere il problema in moduli software più piccoli e indipendenti. Ho preferito abbandonare il controllo centraliz-

zato per organizzare l'architettura in due macro-componenti: un modulo dedicato all'anagrafica (Servizio Utenti) e un modulo focalizzato sulle risorse fisiche (Servizio Colonnine).

Questa separazione non riguarda la sola esecuzione su porte distinte, ma l'intero ciclo di vita del software: ogni servizio opera in modo autonomo e gestisce un proprio strato di persistenza dei dati. Di conseguenza, se il modulo delle colonnine dovesse subire un rallentamento, il sistema sarebbe comunque in grado di consultare l'anagrafica degli utenti senza subire blocchi.

Prima di procedere allo sviluppo, ho formalizzato le interazioni tra l'ambiente esterno e la piattaforma mediante il Diagramma dei Casi d'Uso rappresentato in Figura 2.1.

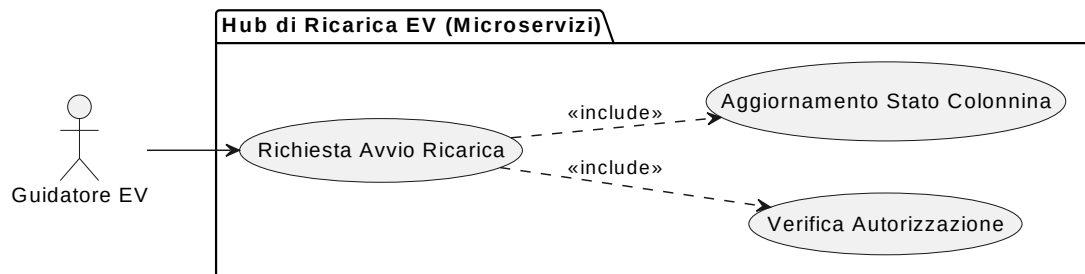


Figura 2.1: Diagramma dei Casi d'Uso: interazione del Guidatore EV con i confini del sistema.

L'attore primario è il Guidatore EV (Electric Vehicle), ossia l'utente finale registrato. L'operazione scatenante è la 'Richiesta Avvio Ricarica', che dal punto di vista ingegneristico si avvale di due relazioni di inclusione («include») per delegare compiti logici specifici.

La prima inclusione, 'Verifica Autorizzazione', è gestita dal Servizio Utenti, che controlla credenziali e stato dell'abbonamento. Se il controllo passa, si attiva la seconda inclusione, 'Aggiornamento Stato Colonnina', affidata al Servizio Colonnine.

È importante notare che il Lock Ottimistico, per quanto cardine della gestione della concorrenza, opera 'sotto il cofano' a livello di persistenza ed è completamente trasparente all'utente finale. Proprio per questo non è stato modellato nel diagramma dei casi d'uso, il quale rappresenta esclusivamente gli obiettivi percepibili dal Guidatore.

## 2.2 Specifica dei Requisiti del Sistema

Per disporre di un tracciato chiaro durante lo sviluppo, ho classificato i vincoli operativi della piattaforma in Requisiti Funzionali (RF) e Non Funzionali (RNF).

### 2.2.1 Requisiti Funzionali

I requisiti funzionali definiscono le azioni specifiche e i comportamenti dinamici che il sistema deve eseguire in risposta agli input degli attori esterni o dei componenti interni:

- **RF1 - Validazione dell'Anagrafica Utente:** Il sistema deve verificare l'esistenza dell'utente e lo stato dell'abbonamento prima di autorizzare la ricarica.
- **RF2 - Gestione dello Stato Operativo delle Colonnine:** Il sistema deve tracciare la disponibilità delle colonnine tramite una macchina a stati finiti (LIBERA, IN\_USO, FUORI\_SERVIZIO).
- **RF3 - Orchestrazione della Richiesta di Ricarica:** Il *Servizio Utenti* funge da gateway per i client e coordina l'operazione transazionale verso il *Servizio Colonnine*.
- **RF4 - Notifica dell'Esito Transazionale:** Il sistema deve restituire messaggi di risposta descrittivi in formato JSON unitamente agli opportuni codici di stato HTTP per riflettere il successo o il fallimento delle operazioni.

### 2.2.2 Requisiti Non Funzionali

I requisiti non funzionali, invece, descrivono le qualità prestazionali, sintattiche e architetturali che la piattaforma deve garantire durante l'esecuzione:

- **RNF1 - Disaccoppiamento e Scalabilità Orizzontale:** La struttura del codice e l'isolamento delle porte di rete devono consentire il deploy e la scalabilità indipendente dei singoli microservizi.
- **RNF2 - Tolleranza alla Concorrenza:** Il sistema deve garantire la consistenza dei dati di fronte a richieste d'accesso simultanee alla medesima risorsa hardware, evitando race condition.
- **RNF3 - Bassa Latenza di Inizializzazione:** L'uso di motori di persistenza in memoria deve consentire tempi di avvio e predisposizione dell'ambiente di test nell'ordine di pochi secondi.

- **RNF4 - Isolamento della Persistenza:** Ciascun microservizio deve gestire le proprie entità all'interno di un'istanza di database dedicata, garantendo un disaccoppiamento architetturale che impedisca query cross-domain dirette.

## 2.3 Pattern di comunicazione e coordinamento distribuito

Scomporre l'applicazione in moduli separati risolve i limiti del monolite, ma introduce inevitabilmente il problema dell'integrazione. Poiché non condividono né memoria centrale né database, il Servizio Utenti e il Servizio Colonnine comunicano esclusivamente via rete. Per mantenere un basso accoppiamento, ho optato per il protocollo HTTP in stile REST, definendo così dei contratti di interfaccia non ambigui.

Nei microservizi, il coordinamento distribuito si basa tipicamente su due pattern: coreografia o orchestrazione. La coreografia, asincrona e guidata dagli eventi (spesso tramite un Message Broker come Kafka o RabbitMQ), permette a ogni servizio di reagire in modo autonomo senza un controllore centrale.

Sebbene offra un disaccoppiamento estremo e ottime prestazioni sotto carico, ho ritenuto inadatto l'uso di un broker per questo progetto. L'assegnazione di una risorsa fisica come la colonnina, infatti, richiede una conferma transazionale immediata. Per questo ho adottato un modello di orchestrazione sincrona.

L'interazione prende il via quando il client invia un payload JSON con una POST all'endpoint del Servizio Utenti (porta 8081). Quest'ultimo, in veste di orchestratore, per portare a termine la transazione, inoltra a sua volta una chiamata HTTP bloccante all'endpoint del Servizio Colonnine (porta 8082), attendendone l'esito prima di rispondere al Guidatore.

Il Diagramma di Sequenza riportato in Figura 2.2, traccia nel dettaglio l'esatto ciclo di vita di questa richiesta di avvio ricarica.

L'analisi del diagramma rivela la complessità nascosta dietro una semplice interazione. Al passo 1, il Guidatore invia la richiesta sulla porta 8081 fornendo il proprio identificativo (`idUtente:"U123"`) e la colonnina desiderata (`idColonnina: 4`). Al passo 2, il Servizio Utenti esegue un controllo puramente locale per validare l'abbonamento. Superato questo step, il modulo anagrafico agisce da client HTTP interno (passo 3) e formula una richiesta POST verso l'endpoint `/api/colonnine/4/occupa` del Servizio Colonnine (porta 8082).

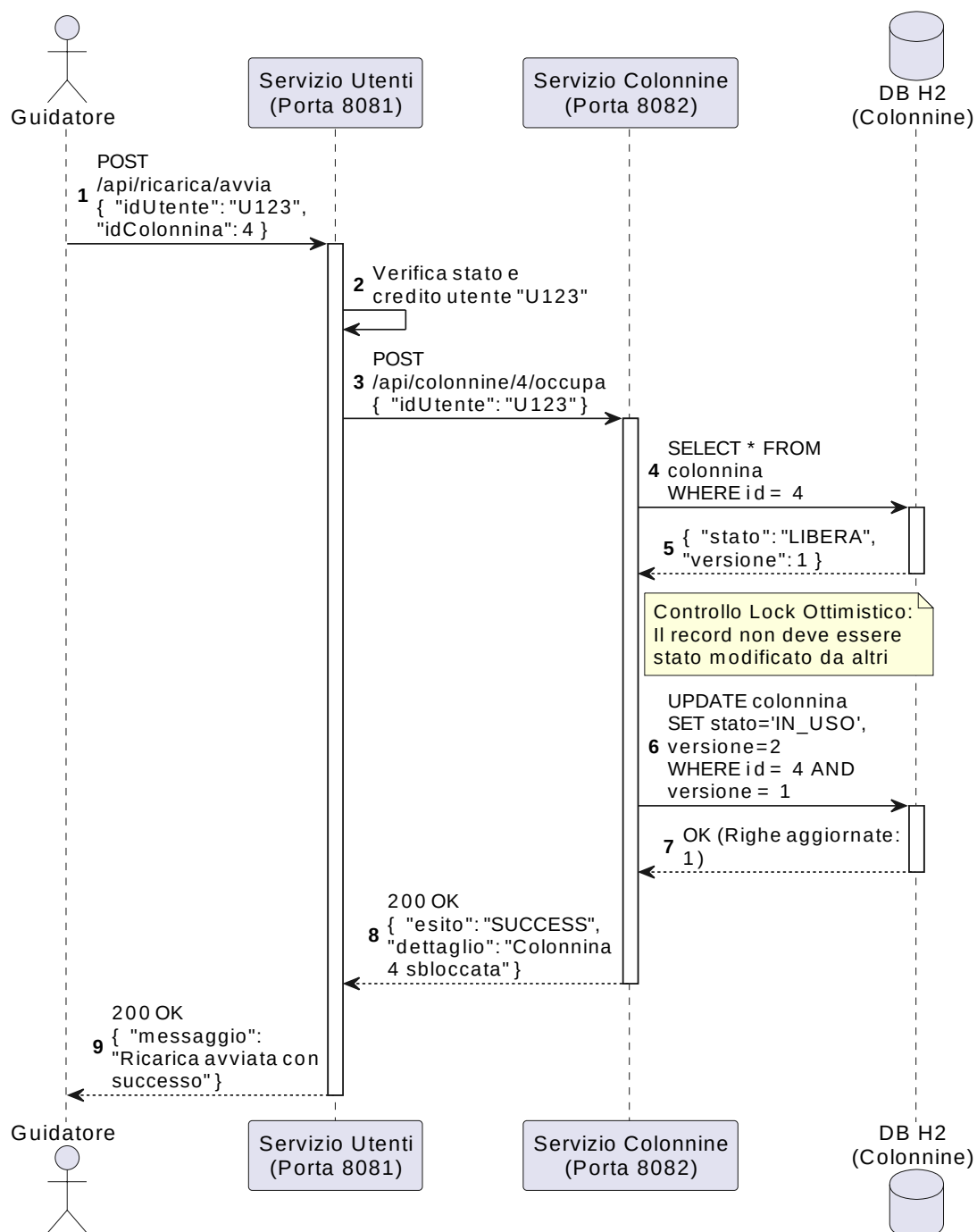


Figura 2.2: Diagramma di Sequenza: orchestrazione della chiamata REST per l'avvio della sessione di ricarica.

Questo livello di separazione è cruciale: il modulo utente ignora del tutto come sia strutturato il parco colonnine. Dal passo 4 in poi, è il Servizio Colonnine a interrogare il proprio database H2 isolato. Dopo aver recuperato lo stato e la *versione* del record (passo 5), effettua il controllo logico e lancia una query di UPDATE condizionale (passo 6). Solo se il database conferma l'aggiornamento della riga (passo 7), l'esito torna indietro. La risposta viaggia sotto forma di *status code* HTTP 200 OK dal Servizio Colonnine al Servizio Utenti (passo 8), che infine prepara il JSON di successo per il client (passo 9).

È importante sottolineare, però, che l'orchestrazione sincrona non è immune ai classici guasti dei sistemi distribuiti. Un caso limite si verifica, ad esempio, quando il Servizio Colonnine applica con successo il lock sul database, ma la risposta HTTP non riesce a tornare al Servizio Utenti a causa di un timeout o di un'interruzione di rete. In questo scenario si creerebbe una grave inconsistenza: l'utente vedrebbe un errore, ma la risorsa fisica resterebbe bloccata in stato `IN_USO`. Per risolvere questa criticità e gestire un vero rollback distribuito, l'evoluzione architetturale dovrà necessariamente integrare il Pattern Saga, avvalendosi di un log degli eventi persistente.

### 2.3.1 Gestione dei Fallimenti Distribuiti: Il Pattern Saga

L'orchestrazione HTTP sincrona, come già accennato, sconta una nota vulnerabilità: l'assenza di un rollback distribuito nativo. Nei database monolitici tradizionali, le proprietà ACID garantiscono l'integrità delle operazioni. Se un'operazione fallisce, il comando `ROLLBACK` del DBMS annulla l'intera transazione.

Tuttavia, nei microservizi questo concetto decade: poiché i database sono fisicamente isolati, non esiste alcun comando che possa eseguire un "rollback globale". Se il Servizio Utenti scala il credito del guidatore (fase 1) ma il Servizio Colonnine va in *timeout* di rete prima di sbloccare la presa (fase 2), il sistema si ritroverebbe in uno stato inconsistente: l'utente avrebbe pagato per una ricarica mai avviata. La risposta accademica e industriale a questo problema è il Pattern Saga, formalizzato da Garcia-Molina e Salem (1987). Una Saga è una sequenza di transazioni locali, dove ogni servizio aggiorna il proprio database e pubblica un evento per innescare il passo successivo. Il punto di forza di questo pattern risiede però nella gestione degli errori: per ogni transazione locale, il sistema deve aver preventivamente codificato una Transazione Compensativa (Compensating Transaction).

Se l'infrastruttura dovesse evolvere verso una Saga Orchestrata (inclusiva del pagamento), il flusso si articolerebbe così:

1. **T1 (Transazione Locale):** Il Servizio Utenti "congela" il credito necessario dal wallet del guidatore.
2. **C1 (Transazione Compensativa):** Sblocco del credito congelato, ripristinando il saldo iniziale.
3. **T2 (Transazione Locale):** Il Servizio Colonnine cambia lo stato della risorsa in `IN_USO`.
4. **C2 (Transazione Compensativa):** Rilascio della colonnina, forzando lo stato a `LIBERA`.

Se la transazione T2 dovesse fallire (ad esempio a causa del Lock Ottimistico in un accesso concorrente), l'orchestratore intercetterebbe il fallimento e lancerebbe in automatico le transazioni compensative in ordine inverso. In questo scenario verrebbe eseguita C1, rimborsando l'utente e ripristinando la Coerenza Eventuale (Eventual Consistency). L'integrazione del Pattern Saga con broker asincroni come Apache Kafka rappresenta quindi il logico e prevedibile passo successivo per portare l'hub a standard produttivi.

### 2.3.2 Il Teorema CAP e i trade-off architetturali

Separare il sistema di ricarica in due servizi fisici che comunicano via rete introduce inevitabilmente le sfide classiche del calcolo distribuito. In questo scenario, ci si deve confrontare con il Teorema CAP, formalizzato da Eric Brewer e dimostrato da Gilbert e Lynch (2002). Il teorema afferma l'impossibilità di garantire simultaneamente tre proprietà:

- **Consistency (Coerenza):** ogni lettura riceve l'aggiornamento più recente o un errore.
- **Availability (Disponibilità):** ogni richiesta riceve una risposta (non di errore), senza però la garanzia che contenga l'informazione più recente.
- **Partition Tolerance (Tolleranza alle partizioni):** il sistema continua a funzionare nonostante la perdita di messaggi o guasti di rete tra i nodi.

Poiché, nei moderni sistemi, la tolleranza alle partizioni (P) non è negoziabile, il progettista è obbligato a compiere un trade-off, scegliendo tra Coerenza (CP) e Disponibilità (AP).

Nel caso specifico di un hub di ricarica, l'assegnazione di una colonnina è un'operazione critica e fortemente transazionale. Assegnare erroneamente la stessa risorsa a due utenti causerebbe un disservizio fisico concreto sul piazzale. Per questo motivo, il sistema progettato si orienta chiaramente verso un modello CP (Coerenza e Tolleranza alle Partizioni). L'utilizzo di una orchestrazione sincrona e bloccante, infatti, fa sì che in caso di indisponibilità del Servizio Colonnine la richiesta di ricarica fallisca completamente. È una scelta progettuale consapevole, che sacrifica la disponibilità assoluta in nome di una coerenza rigorosa dei dati.

## 2.4 Controllo della concorrenza e integrità transazionale

L'architettura distribuita, esposta a richieste simultanee di più utenti, comporta il concreto rischio di incappare in race condition. L'assegnazione di una colonnina fisica è un'operazione particolarmente delicata: se due client tentassero di occupare la stessa risorsa appena liberata, l'assenza di un blocco porterebbe a una doppia assegnazione, compromettendo sia la coerenza dei dati che l'effettivo funzionamento dell'hub.

Per prevenire la corruzione dei dati senza introdurre i costosi rallentamenti dei blocchi pessimistici a livello di tabella, ho adottato una strategia basata sul blocco

ottimistico. Questa tecnica parte dal presupposto che le collisioni al millisecondo nelle operazioni di scrittura siano un evento raro, e predilige quindi la massima fluidità delle letture.

Il meccanismo funziona così: quando il sistema estrae il record della colonnina, salva anche il suo numero di versione corrente. Al momento del commit delle modifiche, il motore relazionale confronta la versione tenuta in memoria con quella attualmente su disco. Se nel transitorio un'altra transazione ha già modificato la colonnina, i numeri non combaceranno e la sovrascrittura verrà respinta. A questo punto viene sollevata un'eccezione, che il livello web intercetta e traduce in una risposta HTTP 409 (Conflict), bloccando l'assegnazione anomala.

Per comprendere appieno le conseguenze di una collisione, il diagramma di sequenza analizzato in precedenza (Figura 2.2) illustra esclusivamente il caso di successo. Per integrare il quadro, descrivo di seguito il failure path, ovvero la sequenza che si innesca quando il controllo di versione fallisce:

1. Il *Servizio Utenti* inoltra la richiesta di occupazione al Servizio Colonnine.
2. Il database estrae lo stato e la versione corrente della colonnina (ad esempio, versione = 1) trattenendola in memoria.
3. A pochi millisecondi dal salvataggio, un'altra transazione concorrente riesce ad aggiornare la medesima colonnina; la versione salvata su disco passa quindi a 2.
4. Il Servizio Colonnine tenta l'UPDATE condizionale con la clausola `WHERE id=4 AND versione=1`.
5. Il database rileva la discrepanza tra la versione in memoria e quella su disco, non aggiorna alcuna riga e solleva un'eccezione.
6. A questo punto, il Servizio Colonnine interrompe l'elaborazione e risponde al Servizio Utenti con un codice di errore HTTP 409 `Conflict`.
7. L'orchestratore annulla l'intera operazione e notifica al guidatore l'errore tramite un JSON, invitandolo a selezionare una colonnina diversa.

## 2.5 Modellazione della persistenza e configurazione degli strati

Il pattern di isolamento dei dati si traduce concretamente nella modellazione a oggetti e nella configurazione infrastrutturale dei due moduli. La scomposizione del dominio, infatti, impone una rigida separazione delle entità, come esplicitato nel Diagramma delle Classi di Figura 2.3.

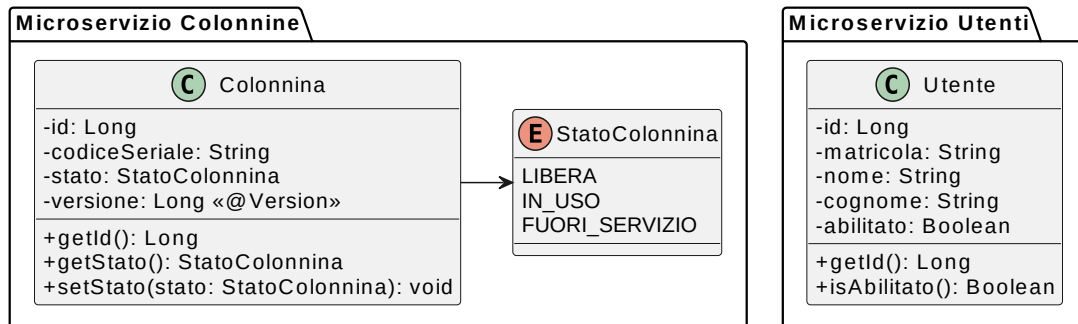


Figura 2.3: Diagramma delle Classi: partizionamento del dominio e isolamento dei dati tra i microservizi.

Il diagramma UML evidenzia volutamente l'assenza di vincoli relazionali (Foreign Key) tra l'anagrafica e l'infrastruttura fisica, una scelta che si discosta nettamente dal design classico dei database monolitici. Il *Microservizio Utenti* incapsula in via esclusiva la classe **Utente**, responsabile di tracciare la **matricola** e il flag booleano **abilitato**. Sul fronte opposto, il *Microservizio Colonnine* mappa la risorsa hardware tramite la classe **Colonnina**. Lo stato operativo di quest'ultima è vincolato a un'enumerazione controllata (**StatoColonnina**), la quale ammette esclusivamente i valori **LIBERA**, **IN\_USO** e **FUORI\_SERVIZIO**.

Dal punto di vista ingegneristico, l'attributo **versione** riveste un ruolo centrale. Annotato con `@Version`, è questo specifico campo a fornire al framework ORM l'ancoraggio necessario per attivare il meccanismo di Optimistic Locking discusso in precedenza.

Per la fase di prototipazione, inizialmente avevo valutato l'uso di un database PostgreSQL esterno. Tuttavia, per snellire l'ambiente di sviluppo e azzerare i tempi di configurazione locale, ho preferito ripiegare su H2 in modalità in-memory. Questa scelta implica un chiaro trade-off: rinunciare alla persistenza permanente dei dati rende il sistema non adatto alla produzione, ma si è rivelata la soluzione più rapida per le finalità di prototipazione e demo richieste da questo lavoro. In questo modo, a ogni avvio i servizi deducono la struttura delle tabelle dalle annotazioni delle classi e le ricostruiscono in pochi secondi. Poiché il database H2 è volatile, i dati vengono persi a ogni spegnimento. Per ovviare a questo e avere un ambiente di test sempre pronto, ho integrato un Data Loader che inietta automaticamente un set di dati di base durante il bootstrap dell'applicazione. Il partizionamento logico descritto nel diagramma delle classi si riflette così in maniera speculare a livello di persistenza fisica.

L'architettura esclude l'uso di chiavi esterne (*Foreign Key*) condivise tra i domini. La struttura relazionale generata all'avvio da Hibernate all'interno delle istanze H2 isolate può essere concettualizzata attraverso il seguente schema Entità-Relazione (E-R):

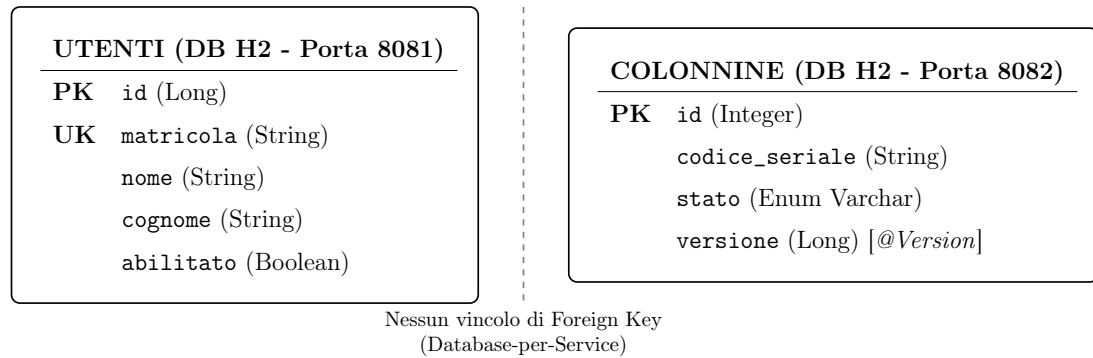


Figura 2.4: Schema E-R: Separazione fisica delle tabelle relazionali sui due database isolati.

Come illustrato in Figura 2.4, l'assenza di un vincolo referenziale tra le due entità obbliga il sistema a gestire le relazioni esclusivamente a livello applicativo, tramite l'orchestrazione sincrona REST descritta nei paragrafi precedenti. Questo approccio azzerà il rischio di lock distribuiti a livello relazionale.

## 2.6 Analisi dei Rischi e Sicurezza delle API

Sebbene lo sviluppo del prototipo sia stato orientato principalmente alla risoluzione della concorrenza e all'integrità transazionale, un sistema reale impone un'analisi dei rischi (Threat Modeling). La criticità più evidente allo stato attuale è l'assenza di un perimetro di autenticazione forte.

Al momento, il Servizio Utenti accetta richieste HTTP POST con un payload JSON in chiaro dove l'identificativo del guidatore è una semplice stringa (ad esempio, `idUtente:"U123"`). Questa modalità espone il sistema a una vulnerabilità critica: lo spoofing. Un utente malintenzionato potrebbe intercettare il traffico non cifrato o, con un attacco a forza bruta, indovinare la matricola di un utente abbonato. Così facendo, sarebbe in grado di avviare o interrompere ricariche a nome di altri, bypassando completamente le policy di sicurezza.

Per mitigare il rischio, il paradigma microservizi suggerisce di esternalizzare il controllo degli accessi. La soluzione definitiva prevede l'integrazione di un Identity Provider (come Keycloak) e l'adozione del protocollo OAuth2. Con OAuth2, l'app mobile del client non invia più la matricola in chiaro. Dopo il login, ottiene

un JSON Web Token (JWT) firmato crittograficamente, da inserire nell'header HTTP. In questa configurazione avanzata, il microservizio non si fida ciecamente dei dati esterni: estrae l'identificativo utente dai claim del token, garantendo l'autenticità e l'integrità del mittente.

## 2.7 Conclusioni del Capitolo

L'analisi progettuale condotta in questo capitolo ha delineato un perimetro architetturale solido per l'hub di ricarica. La spinta a superare i limiti del monolite ha portato alla scomposizione del dominio, evitando che il ciclo di vita di una risorsa fisica fosse rigido e accoppiato ai processi anagrafici. Questa separazione delle responsabilità, pilastro dei microservizi, si riflette in una maggiore resilienza e scalabilità.

Le sfide dei sistemi distribuiti hanno richiesto un'attenta valutazione dei trade-off ingegneristici. Il Teorema CAP ha guidato la scelta di privilegiare la coerenza assoluta a scapito della disponibilità continua; il Lock Ottimistico ha messo al sicuro l'integrità dei dati contro gli accessi concorrenti. Inoltre, l'analisi preventiva delle minacce (Threat Modeling) ha già tracciato la via per l'implementazione di standard di sicurezza futuri.

La modellazione UML dei comportamenti e delle comunicazioni ha fornito le specifiche rigorose necessarie. Il prossimo capitolo entrerà nel vivo dell'implementazione pratica, analizzando i framework e le logiche che hanno tradotto questa architettura concettuale in microservizi operativi.

# Capitolo 3

## Tecnologie e Strumenti Utilizzati

Prima di approfondire l'implementazione, è necessario definire e motivare lo stack tecnologico adottato. Le scelte in questa fase sono state orientate a bilanciare la flessibilità tipica dei microservizi con la robustezza transazionale che un sistema di prenotazione di risorse fisiche richiede.

### 3.1 Ecosistema Java e Spring Boot

Per lo sviluppo dei microservizi ho scelto Java (versione 17) con Spring Boot (versione 3.x).

Inizialmente avevo valutato stack più leggeri e asincroni come Node.js con Express. Sebbene Node.js sia ottimo nella gestione di elevati carichi di I/O grazie al suo Event Loop, la sua natura asincrona complica la gestione nativa dei lock concorrenti e delle transazioni relazionali stringenti. Al contrario, Spring Boot offre un ecosistema enterprise maturo e integra nativamente strumenti avanzati di sicurezza e gestione dei dati.

Un altro vantaggio decisivo è il suo approccio 'opinionated'. Spring Boot fornisce un server web Tomcat integrato (embedded), che consente di compilare ogni microservizio come un processo isolato. Ho ritenuto questa caratteristica ideale per eliminare la necessità di complessi application server esterni, allineandomi al principio architetturale per cui i microservizi devono essere indipendenti e facilmente containerizzabili.

## 3.2 Persistenza dei Dati: Spring Data JPA e H2 Database

Per la persistenza, ho evitato di scrivere query SQL manuali (JDBC vanilla), optando invece per Spring Data JPA. Questo modulo offre un'astrazione di alto livello e si interfaccia con Hibernate, l'ORM provider predefinito. Ho scelto questo approccio per mappare le tabelle relazionali in classi Java (Entity) e per demandare al framework la generazione automatica dello schema DDL all'avvio.

### 3.2.1 Dietro le quinte di Hibernate: la traduzione SQL di @Version

Come già accennato, la gestione della concorrenza è stato lo scoglio progettuale principale. Hibernate offre una soluzione elegante grazie all'annotazione `@Version`.

Quando il Servizio Colonnine elabora una richiesta di sblocco, Hibernate traduce l'aggiornamento dello stato dell'entità in una precisa istruzione SQL condizionale. Se assumiamo che l'utente tenti di prenotare la colonnina con `id=4` e versione corrente 1, l'ORM non genererà un semplice `UPDATE`, ma bensì:

```
1 UPDATE colonnine
2 SET stato = 'IN_USO', versione = 2
3 WHERE id = 4 AND versione = 1;
```

Listing 3.1: Traduzione SQL generata da Hibernate per il Lock Ottimistico

La potenza di questo approccio risiede nell'operatore logico `AND`. Il database verifica se il record ha ancora versione 1. Se nel frattempo un'altra transazione ha modificato il record, la versione su disco sarà 2. In tal caso, la clausola `WHERE` fallisce, nessuna riga viene aggiornata, e Hibernate lancia `ObjectOptimisticLockingFailureException`. Questa logica, semplice ma efficace, garantisce l'integrità dei dati senza i costi di performance dei bloccanti lock di riga (come `SELECT ... FOR UPDATE`).

### 3.2.2 Scelte infrastrutturali: H2 Database vs Testcontainers

Per il motore di persistenza ho scelto H2 Database in modalità in-memory. Inizialmente la scelta è ricaduta su H2 per i suoi innegabili vantaggi in ambito prototipale: la creazione di database isolati a runtime, la rapidità del Data Loader e la comoda console web di ispezione.

Tuttavia, una valutazione più critica rivela che un database in memoria rappresenta un limite per un sistema cloud-native di livello Enterprise. H2 non riproduce le latenze fisiche di I/O su disco, nasconde le complessità della gestione del connection pool (es. HikariCP) e utilizza un dialetto SQL diverso da quello dei DBMS relazionali di produzione. Questo divario comporta un rischio concreto: i test potrebbero passare in locale per poi fallire in fase di deploy in ambiente di staging.

La soluzione moderna a questa discrepanza è rappresentata da Testcontainers, ormai integrata nativamente in Spring Boot. Questa libreria permette di istanziare veri container Docker 'usa e getta' (ad esempio con PostgreSQL) durante i test di integrazione, distruggendoli al termine. In tal modo, il codice viene collaudato sull'esatto motore che andrà in produzione.

Per questa tesi ho mantenuto H2 per garantire un prototipo leggero e compilabile su qualsiasi macchina, ma l'adozione di Testcontainers rappresenta il naturale passo successivo e lo standard industriale per integrare il progetto in pipeline CI/CD aziendali.

Per fare in modo che H2 funzionasse correttamente in modalità *in-memory* per ciascun servizio isolato, ho dovuto configurare attentamente il file `application.properties` in entrambi i moduli. Ad esempio, per il Servizio Colonnine l'impostazione è la seguente:

```
1 spring.datasource.url=jdbc:h2:mem:colonninedb
2 spring.datasource.driverClassName=org.h2.Driver
3 spring.datasource.username=sa
4 spring.datasource.password=
5 spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
6
7 # Abilitazione console per ispezione dati
8 spring.h2.console.enabled=true
9 spring.h2.console.path=/h2-console
```

Listing 3.2: Configurazione `application.properties` per il database H2

### 3.3 Gestione delle Dipendenze e Build: Apache Maven

Ho affidato ad Apache Maven l'intera gestione del ciclo di vita del software, dalla risoluzione delle dipendenze alla compilazione. Durante il setup avevo considerato anche Gradle, ma ho preferito Maven perché il suo standard XML dichiarativo offre una curva di apprendimento ben consolidata e si integra in modo nativo e trasparente con l'ecosistema Spring Boot.

Dal punto di vista architetturale, ho evitato di creare un progetto multi-modulo centralizzato con un parent POM condiviso. Ho optato deliberatamente per due file pom.xml separati e indipendenti. Questa scelta non è casuale: rispetta rigorosamente il principio di indipendenza assoluta del deploy nei microservizi. Grazie a questo isolamento a livello di build, una modifica o un aggiornamento delle dipendenze nel Servizio Utenti non può causare conflitti o forzare la ricompilazione del Servizio Colonnine.

Oltre a risolvere i conflitti di versione, Maven è stato sfruttato per automatizzare l'intero build lifecycle. Grazie al plugin spring-boot-maven-plugin, il comando package non solo compila il codice ed esegue i test, ma esegue anche il repackaging: l'applicazione viene impacchettata in un fat-JAR (o uber-JAR). Questo eseguibile contiene al suo interno le classi del progetto, il server Tomcat embedded e tutte le dipendenze.

Il microservizio generato è quindi completamente standalone e pronto per essere avviato su qualsiasi server tramite un semplice comando Java. Questo mi ha permesso di eliminare la necessità di configurare application server esterni, gettando le basi ideali per una futura containerizzazione tramite Docker.

# Capitolo 4

## Implementazione dei Microservizi con Spring Boot

### 4.1 Modellazione e Sviluppo del Servizio Utenti

Il primo passo nell'implementazione pratica ha riguardato il *Servizio Utenti*, il modulo incaricato di gestire l'anagrafica e, soprattutto, di fungere da scudo autorizzativo per l'intera piattaforma.

Partendo dal Diagramma delle Classi definito in fase di design, ho mappato l'entità principale utilizzando le annotazioni della Java Persistence API (JPA). L'obiettivo era far sì che Hibernate, il provider ORM di default in Spring Boot, generasse automaticamente lo schema del database all'avvio.

Ecco l'implementazione della classe *Utente*:

```
1 package it.vanvitelli.utenti.model;
2
3 import jakarta.persistence.*;
4
5 @Entity
6 @Table(name = "utenti")
7 public class Utente {
8
9     @Id
10     @GeneratedValue(strategy = GenerationType.IDENTITY)
11     private Long id;
12
13     @Column(unique = true, nullable = false, length = 50)
14     private String matricola;
15
16     @Column(nullable = false)
17     private String nome;
```

```

18
19     @Column(nullable = false)
20     private String cognome;
21
22     @Column(nullable = false)
23     private boolean abilitato;
24
25     // Costruttore vuoto richiesto da JPA
26     public Utente() {}
27
28     public Utente(String matricola, String nome, String cognome,
29         boolean abilitato) {
30         this.matricola = matricola;
31         this.nome = nome;
32         this.cognome = cognome;
33         this.abilitato = abilitato;
34     }
35
36     // Getters
37     public Long getId() { return id; }
38     public String getMatricola() { return matricola; }
39     public String getNome() { return nome; }
40     public String getCognome() { return cognome; }
41     public boolean isAbilitato() { return abilitato; }
42
43     // Setters omissi per brevità

```

Listing 4.1: Classe Entity Utente.java

Come si nota dal listato, ho impostato il vincolo `unique = true` sulla colonna `matricola`. Ho ritenuto questa precauzione fondamentale perché la `matricola` (ad esempio "U123") è il parametro chiave nei payload JSON inviati dai client; dovevo garantire a livello di database l'impossibilità di avere due profili con lo stesso identificativo, prevenendo bug logici in fase di autorizzazione. L'attributo `abilitato`, invece, rappresenta lo stato dell'abbonamento ed è il perno attorno a cui ruota la logica di sblocco della ricarica.

Per l'accesso ai dati, invece di scrivere le classiche query SQL manuali, ho sfruttato la potenza di Spring Data JPA. Mi è bastato definire un'interfaccia che estendesse `JpaRepository` per ereditare automaticamente tutti i metodi CRUD standard.

```

1 package it.vanvitelli.utenti.repository;
2
3 import it.vanvitelli.utenti.model.Utente;

```

```

4 import org.springframework.data.jpa.repository.JpaRepository;
5 import org.springframework.stereotype.Repository;
6 import java.util.Optional;
7
8 @Repository
9 public interface UtenteRepository extends JpaRepository<Utente,
    Long> {
10
11     // Metodo custom generato dinamicamente da Spring
12     Optional<Utente> findByMatricola(String matricola);
13 }

```

Listing 4.2: Interfaccia UtenteRepository.java

In questa interfaccia ho dichiarato esplicitamente il solo metodo `findByMatricola`. Sfruttando la risoluzione dinamica dei nomi di Spring, il framework traduce la firma del metodo in una query SQL (`SELECT * FROM utenti WHERE matricola = ?`) in totale autonomia a runtime. Inoltre, ho utilizzato il wrapper `Optional` (introdotta in Java 8) per gestire elegantemente i casi in cui la matricola fornita dal guidatore non esista nel database, evitando così il rischio di *NullPointerException* durante l'elaborazione della richiesta.

#### 4.1.1 Esposizione delle API e Orchestrazione REST

Definito lo strato di persistenza, ho dovuto creare un punto di accesso per far dialogare l'applicazione mobile del guidatore con il sistema. Per questo ho sviluppato `UtenteController`, un componente `@RestController` incaricato di ricevere le richieste HTTP, validare le regole di business e, soprattutto, orchestrare la chiamata verso il Microservizio Colonnine.

Per gestire la comunicazione tra i due servizi ho implementato `RestTemplate`, il client HTTP standard di Spring. So bene che la documentazione ufficiale considera ormai questo strumento in maintenance mode, suggerendo alternative più moderne come `WebClient` o il recente `RestClient`.

Tuttavia, ho mantenuto `RestTemplate` per una ragione architetturale precisa e consapevole: l'assegnazione della risorsa richiede un flusso rigorosamente sincrono e bloccante. Il Servizio Utenti, in veste di orchestratore, deve attendere l'esito effettivo della transazione sul database remoto prima di poter rispondere al cliente finale. In questo contesto, l'uso di costrutti asincroni sarebbe stato un'aggiunta superflua e non allineata con l'obiettivo primario di consistenza.

Di seguito è riportato lo snippet iniziale che rappresenta il cuore della logica di orchestrazione:

```
1 package it.vanvitelli.utenti.controller;
2
3 import it.vanvitelli.utenti.model.Utente;
4 import it.vanvitelli.utenti.repository.UtenteRepository;
5 import org.springframework.beans.factory.annotation.Autowired;
6 import org.springframework.http.*;
7 import org.springframework.web.bind.annotation.*;
8 import org.springframework.web.client.HttpClientErrorException;
9 import org.springframework.web.client.RestTemplate;
10
11 import java.util.Map;
12 import java.util.Optional;
13
14 @RestController
15 @RequestMapping("/api/ricarica")
16 public class UtenteController {
17
18     @Autowired
19     private UtenteRepository utenteRepository;
20
21     @Autowired
22     private RestTemplate restTemplate;
23
24     private final String COLONNINE_URL =
25         "http://localhost:8082/api/colonnine";
26
27     @PostMapping("/avvia")
28     public ResponseEntity<?> avviaRicarica(
29         @RequestBody Map<String, Object> payload) {
30
31         String matricola = (String) payload.get("idUtente");
32         Integer idCol = (Integer) payload.get("idColonnina");
33
34         // 1. Validazione dell'utente
35         Optional<Utente> utenteOpt =
36             utenteRepository.findByMatricola(matricola);
37
38         if (utenteOpt.isEmpty()) {
39             return ResponseEntity.status(HttpStatus.NOT_FOUND)
40                 .body(Map.of("errore", "Utente non trovato"));
41         }
42
43     }
```

```

43     Utente utente = utenteOpt.get();
44     if (!utente.isAbilitato()) {
45         return ResponseEntity.status(HttpStatus.FORBIDDEN)
46             .body(Map.of(
47                 "errore", "Abbonamento disabilitato"
48             ));
49     }
50
51     // 2. Chiamata REST al Servizio Colonnine
52     String url = COLONNINE_URL + "/" + idCol + "/occupa";
53
54     HttpHeaders headers = new HttpHeaders();
55     headers.setContentType(MediaType.APPLICATION_JSON);
56
57     HttpEntity<Map<String, String>> requestEntity =
58         new HttpEntity<>(
59             Map.of("idUtente", matricola),
60             headers
61         );
62     try {
63         ResponseEntity<Map> response =
64             restTemplate.postForEntity(
65                 url, requestEntity, Map.class
66             );
67         return ResponseEntity.ok(Map.of(
68             "messaggio", "Ricarica avviata",
69             "dettaglio", response.getBody()
70         ));
71     } catch (HttpClientErrorException.Conflict e) {
72         // 3. Gestione dell'errore di concorrenza
73         return ResponseEntity.status(HttpStatus.CONFLICT)
74             .body(Map.of("errore", "Colonnina gia' occupata"));
75     };
76     } catch (Exception e) {
77         return ResponseEntity.status(
78             HttpStatus.INTERNAL_SERVER_ERROR
79         ).body(Map.of("errore", "Errore di comunicazione"));
80     }
81 }

```

Listing 4.3: Classe UtenteController.java

Se vado ad osservare questo snippet, è possibile osservare come ho concretizzato nel codice i requisiti architetturali definiti in precedenza. In una prima fase verifico l'esistenza e l'abilitazione del profilo estraendolo dal database H2 locale.

Solo se i controlli passano, il servizio cambia "cappello": da server diventa client HTTP e formatta una richiesta POST indirizzata alla porta 8082.

La gestione delle eccezioni tramite i blocchi `try-catch` rappresenta uno dei punti cardine di questa classe. Intercettando specificamente `HttpClientErrorException.Conflict`, il sistema è in grado di riconoscere se il Microservizio Colonnine ha bloccato l'operazione a causa dell'intervento del *Lock Ottimistico* (il flusso di errore analizzato nel capitolo precedente). In quel caso, il controller del modulo utenti "ammortizza" l'errore del server interno e restituisce all'applicazione del guidatore un messaggio descrittivo e pulito in formato JSON.

#### 4.1.2 Data Transfer Object (DTO) e Validazione dell'Input

Come mostrato nello snippet precedente, nella prima iterazione dello sviluppo gestivo i payload HTTP tramite generiche `Map<String, Object>`. Per innalzare la robustezza dell'applicazione, ho successivamente effettuato un refactoring, introducendo il pattern Data Transfer Object (DTO) nel `UtenteController`.

Utilizzare un DTO dedicato permette di disaccoppiare il modello di dominio (Entity) dai dati esposti dalle API. Inoltre, integrando la dipendenza `spring-boot-starter-validation`, ho potuto demandare al framework la validazione dell'input attraverso annotazioni dichiarative.

```
1 package it.vanvitelli.utenti.dto;
2
3 import jakarta.validation.constraints.NotBlank;
4 import jakarta.validation.constraints.NotNull;
5 import jakarta.validation.constraints.Pattern;
6
7 public class RicaricaRequestDTO {
8
9     @NotBlank(message = "L'identificativo utente non puo' essere vuoto")
10    @Pattern(regexp = "^U\\d+$", message = "Formato matricola non valido (es. U123)")
11    private String idUtente;
12
13    @NotNull(message = "L'ID della colonnina e' obbligatorio")
14    private Integer idColonnina;
15
16    // Costruttore vuoto
17    public RicaricaRequestDTO() {}
18
19    // Getters e Setters
```

```

20     public String getIdUtente() { return idUtente; }
21     public void setIdUtente(String idUtente) { this.idUtente =
        idUtente; }
22
23     public Integer getIdColonnina() { return idColonnina; }
24     public void setIdColonnina(Integer idColonnina) { this.
        idColonnina = idColonnina; }
25 }

```

Listing 4.4: Implementazione della classe RicaricaRequestDTO con validazione

Di conseguenza, la firma del metodo nel controller è stata aggiornata sostituendo la mappa generica con il nuovo DTO, preceduto dall'annotazione `@Valid`:

```

1 @PostMapping("/avvia")
2 public ResponseEntity<?> avviaRicarica(@Valid @RequestBody
    RicaricaRequestDTO payload) {
3     String matricola = payload.getIdUtente();
4     Integer idCol = payload.getIdColonnina();
5
6     // ... esecuzione della logica di validazione abbonamento ...
7 }

```

Listing 4.5: Aggiornamento della firma del Controller con `@Valid`

Grazie a questo approccio, se un cliente invia una richiesta senza l'ID colonnina o con una matricola malformata (ad esempio, con lettere al posto dei numeri), Spring Boot intercetta l'anomalia prima di eseguire la logica di business. Il framework restituisce così un errore 400 `Bad Request` corredato dal messaggio descrittivo definito nelle annotazioni.

### 4.1.3 Documentazione Contratti REST: OpenAPI e Swagger

In un'architettura a microservizi, dove la comunicazione avviene esclusivamente tramite rete, il "contratto" formale delle richieste e risposte assume un'importanza vitale. Non è sufficiente che il codice funzioni; è indispensabile che gli endpoint siano documentati in modo standardizzato, così che i client sappiano esattamente come formattare i payload JSON e quali risposte attendersi.

Per soddisfare questo requisito, ho evitato la documentazione manuale statica (soggetta a rapida obsolescenza), optando invece per la specifica OpenAPI 3.0 tramite la libreria `springdoc-openapi`.

Aggiungendo questa dipendenza, Spring Boot espone automaticamente l'interfaccia Swagger UI all'indirizzo `/swagger-ui.html`. Per una documentazione più

ingegneristica e dettagliata, ho arricchito i controller con le annotazioni native di Swagger, come mostrato di seguito per l'endpoint di avvio ricarica:

```
1 @Operation(  
2     summary = "Avvia una nuova sessione di ricarica",  
3     description = "Verifica l'abilitazione dell'utente e instrada  
4         la transazione di occupazione verso il Servizio Colonnine."  
5 )  
6 @ApiResponses(value = {  
7     @ApiResponse(responseCode = "200", description = "Ricarica  
8         avviata con successo"),  
9     @ApiResponse(responseCode = "403", description = "Utente  
10         disabilitato o abbonamento scaduto"),  
11     @ApiResponse(responseCode = "404", description = "Utente non  
12         trovato nel database locale"),  
13     @ApiResponse(responseCode = "409", description = "Conflitto:  
14         la colonnina fisica risulta già' occupata")  
15 })  
16 @PostMapping("/avvia")  
17 public ResponseEntity<?> avviaRicarica(@RequestBody Map<String,  
18     Object> payload) {  
19     // ... logica del controller ...  
20 }
```

Listing 4.6: Annotazioni OpenAPI per la documentazione dell'endpoint

Questo approccio dichiarativo garantisce che il codice e la sua documentazione rimangano sempre allineati (Single Source of Truth), eliminando alla radice il rischio di disallineamenti tra il backend e i client che ne consumano le API.

## 4.2 Sviluppo del Servizio Colonnine e Optimistic Locking

Il secondo pilastro dell'architettura è il Servizio Colonnine, incaricato di monitorare e assegnare le risorse fisiche. In questo dominio, la sfida ingegneristica più complessa non è stata la modellazione dei dati, bensì la gestione della concorrenza. Per risolvere il problema della doppia prenotazione, ho implementato un meccanismo di Lock Ottimistico a livello di database, sfruttando le annotazioni JPA. Di seguito, la struttura dell'entità Colonnina:

```
1 package it.vanvitelli.colonnine.model;  
2  
3 import jakarta.persistence.*;  
4
```

```

5 @Entity
6 @Table(name = "colonnine")
7 public class Colonnina {
8
9     @Id
10    @GeneratedValue(strategy = GenerationType.IDENTITY)
11    private Integer id;
12
13    @Column(nullable = false)
14    private String codiceSeriale;
15
16    // Type safety garantita dall'Enum
17    @Enumerated(EnumType.STRING)
18    @Column(nullable = false)
19    private StatoColonnina stato;
20
21    // Il cuore della gestione della concorrenza
22    @Version
23    private Long versione;
24
25    public Colonnina() {}
26
27    public Colonnina(String codiceSeriale, StatoColonnina stato)
28    {
29        this.codiceSeriale = codiceSeriale;
30        this.stato = stato;
31    }
32
33    // Getters
34    public Integer getId() { return id; }
35    public String getCodiceSeriale() { return codiceSeriale; }
36    public StatoColonnina getStato() { return stato; }
37    public Long getVersion() { return versione; }
38
39    // Setters
40    public void setStato(StatoColonnina stato) { this.stato =
    stato; }
41 }

```

Listing 4.7: Classe Entity Colonnina.java

L'annotazione `@Version` è l'elemento cardine di questa entità. Tramite essa, Hibernate inietta automaticamente un controllo di versione in ogni operazione di UPDATE. Quando il sistema cerca di portare una colonnina da LIBERA a IN\_USO, il framework verifica che la versione in memoria corrisponda a quella nel database.

Se un altro thread (un'altra richiesta concorrente) ha già modificato il record, la versione su disco risulterà superiore: la transazione corrente solleva allora un'eccezione `ObjectOptimisticLockingFailureException`. Questo meccanismo garantisce l'integrità assoluta dei dati senza i costi prestazionali dei lock pessimistici di riga o di tabella, che avrebbero penalizzato la scalabilità dell'intero hub. Per completare il modulo di persistenza, ho creato il repository estendendo `JpaRepository`. La struttura astratta di Spring Data mi ha permesso di evitare la scrittura manuale di query boilerplate.

```
1 package it.vanvitelli.colonnine.repository;
2 import it.vanvitelli.colonnine.model.Colonnina;
3 import org.springframework.data.jpa.repository.JpaRepository;
4 import org.springframework.stereotype.Repository;
5
6 @Repository
7 public interface ColonninaRepository extends JpaRepository<
8     Colonnina, Integer> {
9     // Eredita tutti i metodi standard di persistenza ed
10    elaborazione
11 }
```

Listing 4.8: Interfaccia `ColonninaRepository.java`

### 4.2.1 Disaccoppiamento della Logica di Business: Il Service Layer

Nelle fasi di prototipazione iniziale, la logica transazionale risiedeva a livello di Controller. Tuttavia, le best practice del pattern MVC impongono un disaccoppiamento più netto: il Controller dovrebbe gestire solo la validazione dell'input e la risposta HTTP, delegando le regole di business a uno strato intermedio. Per allinearmi a questo standard Enterprise, ho introdotto la classe `ColonninaService`, annotata con `@Service`. Questa componente incapsula la logica di sblocco e, tramite `@Transactional`, delimita il perimetro delle transazioni di database.

```
1 package it.vanvitelli.colonnine.service;
2
3 import it.vanvitelli.colonnine.model.Colonnina;
4 import it.vanvitelli.colonnine.model.StatoColonnina;
5 import it.vanvitelli.colonnine.repository.ColonninaRepository;
6 import org.springframework.beans.factory.annotation.Autowired;
7 import org.springframework.stereotype.Service;
8 import org.springframework.transaction.annotation.Transactional;
9 import java.util.Optional;
```

```

10
11 @Service
12 public class ColonninaService {
13
14     @Autowired
15     private ColonninaRepository colonninaRepository;
16
17     @Transactional
18     public Colonnina occupaColonnina(Integer id) {
19         Optional<Colonnina> colOpt = colonninaRepository.findById
20         (id);
21
22         if (colOpt.isEmpty()) {
23             throw new IllegalArgumentException("Colonnina non
24             trovata");
25         }
26
27         Colonnina colonnina = colOpt.get();
28
29         // Controllo type-safe dello stato
30         if (colonnina.getStato() != StatoColonnina.LIBERA) {
31             throw new IllegalStateException("Colonnina gia'
32             occupata o fuori servizio");
33         }
34         // Il cambio di stato
35         colonnina.setStato(StatoColonnina.IN_USO);
36
37         // Il salvataggio scatena automaticamente il controllo di
38         @Version
39         return colonninaRepository.save(colonnina);
40     }
41 }

```

Listing 4.9: Implementazione del Service Layer per la gestione transazionale

L'uso di `@Transactional` è cruciale in questo contesto: garantisce che l'intera operazione di lettura, controllo e aggiornamento avvenga all'interno della medesima transazione logica sul database. Se per qualsiasi motivo (incluso il fallimento del *Lock Ottimistico*) viene sollevata un'eccezione, il framework Spring si occupa di eseguire automaticamente il *rollback*, riportando il database allo stato integro precedente. A seguito di questo *refactoring*, il `ColonninaController` si è trasformato in un componente snello ("*Thin Controller*"), il cui unico scopo è richiamare il `ColonninaService` e smistare il risultato al client.

## 4.2.2 Logica di Business e Gestione del Conflitto HTTP 409

La parte più delicata del Servizio Colonnine risiede nel suo `ColonninaController`. È qui che si concretizza l'endpoint invocato dal Servizio Utenti. Quando arriva la richiesta di occupazione, il sistema verifica se la colonnina è libera, ne modifica lo stato e tenta il salvataggio.

Se due richieste concorrenti arrivano nello stesso istante, una delle due fallirà a causa del meccanismo di `@Version` analizzato in precedenza. In quel momento, Spring solleva un'eccezione che il controller intercetta, traducendola nell'apposito codice di stato *HTTP 409 Conflict*.

Di seguito riporto l'implementazione del controller:

```
1 package it.vanvitelli.colonnine.controller;
2
3 import it.vanvitelli.colonnine.model.Colonnina;
4 import it.vanvitelli.colonnine.model.StatoColonnina;
5 import it.vanvitelli.colonnine.repository.ColonninaRepository;
6 import org.springframework.beans.factory.annotation.Autowired;
7 import org.springframework.http.*;
8 import org.springframework.orm.*;
9 import org.springframework.web.bind.annotation.*;
10 import java.util.Map;
11 import java.util.Optional;
12
13 @RestController
14 @RequestMapping("/api/colonnine")
15 public class ColonninaController {
16
17     @Autowired
18     private ColonninaRepository colonninaRepository;
19
20     @PostMapping("/{id}/occupa")
21     public ResponseEntity<?> occupaColonnina(
22         @PathVariable Integer id,
23         @RequestBody Map<String, String> payload) {
24         Optional<Colonnina> colOpt =
25             colonninaRepository.findById(id);
26         if (colOpt.isEmpty()) {
27             return ResponseEntity.status(HttpStatus.NOT_FOUND)
28                 .body(Map.of("errore", "Colonnina inesistente
29         ));
30         }
31         Colonnina colonnina = colOpt.get();
```

```

31     // Controllo type-safe tramite Enum
32     if (colonnina.getStato() != StatoColonnina.LIBERA) {
33         return ResponseEntity.status(HttpStatus.CONFLICT)
34             .body(Map.of("errore", "Colonnina occupata"));
35     }
36     // Tentativo di aggiornamento dello stato
37     colonna.setStato(StatoColonnina.IN_USO);
38     try {
39         // Il salvataggio scatena il controllo
40         // del Lock Ottimistico
41         colonnaRepository.save(colonna);
42
43         return ResponseEntity.ok(Map.of(
44             "idColonnina", colonna.getId(),
45             "stato", colonna.getStato().name(),
46             "seriale", colonna.getCodiceSeriale()
47         ));
48     } catch (ObjectOptimisticLockingFailureException e) {
49         // Conflitto rilevato: versione obsoleta in memoria
50         return ResponseEntity.status(HttpStatus.CONFLICT)
51             .body(Map.of(
52                 "errore",
53                 "Conflitto: risorsa modificata " +
54                 "da un'altra transazione."
55             ));
56     }
57 }
58 }

```

Listing 4.10: Classe ColonninaController.java

Questo blocco di codice completa l'implementazione della logica transazionale del sistema. Grazie al blocco `catch` mirato sull'eccezione `ObjectOptimisticLockingFailureException`, il microservizio garantisce che nessun utente possa sovrascrivere lo stato di una colonna già assegnata, restituendo un feedback pulito e standardizzato verso l'esterno.

### 4.2.3 Refactoring Architettuale: Gestione Centralizzata delle Eccezioni

Nelle prime fasi di prototipazione, la gestione dei conflitti di concorrenza è stata implementata localmente all'interno dei singoli *Controller* tramite l'utilizzo di blocchi `try-catch`. Sebbene questo approccio sia funzionale per testare rapida-

mente l'efficacia del *Lock Ottimistico*, esso presenta due gravi limiti strutturali: viola il principio della *Separation of Concerns* (Separazione delle Responsabilità) e porta inevitabilmente alla duplicazione del codice (*boilerplate*) all'aumentare degli *endpoint*.

In un'architettura distribuita seria, è imperativo che tutti i microservizi restituiscano messaggi di errore strutturati secondo un formato JSON rigoroso e standardizzato. Un *API Gateway* o un *client frontend* devono sapere esattamente in quali campi cercare il dettaglio dell'errore, senza rischiare di rompere la decodifica.

Per risolvere questo problema architetturico, ho effettuato un *refactoring* introducendo un meccanismo di gestione globale delle eccezioni, sfruttando il pattern *AOP* (*Aspect-Oriented Programming*) offerto nativamente da Spring Boot tramite l'annotazione `@RestControllerAdvice`.

Il primo passo è stato definire un *Data Transfer Object* (DTO) immutabile, progettato esclusivamente per formattare e incapsulare la risposta di errore:

```
1 package it.vanvitelli.colonnine.dto;
2 import java.time.LocalDateTime;
3 public class ErrorResponse {
4     private final String timestamp;
5     private final int status;
6     private final String error;
7     private final String message;
8     public ErrorResponse(int status, String error, String message
9         ) {
10         this.timestamp = LocalDateTime.now().toString();
11         this.status = status;
12         this.error = error;
13         this.message = message;
14     }
15     // Getters omissi per brevità
16 }
```

Listing 4.11: Struttura del DTO standardizzato per gli errori

Successivamente, ho rimosso la gestione esplicita degli errori dai controller (rendendo il codice molto più pulito e focalizzato solo sul *happy path*) e ho delegato l'intercettazione a un componente centralizzato: il `GlobalExceptionHandler`.

```
1 package it.vanvitelli.colonnine.exception;
2
3 import it.vanvitelli.colonnine.dto.ErrorResponse;
4 import org.springframework.http.HttpStatus;
5 import org.springframework.http.ResponseEntity;
```

```

6 import org.springframework.orm.
    ObjectOptimisticLockingFailureException;
7 import org.springframework.web.bind.annotation.ExceptionHandler;
8 import org.springframework.web.bind.annotation.
    RestControllerAdvice;
9
10 @RestControllerAdvice
11 public class GlobalExceptionHandler {
12
13     // Intercetta la violazione del Lock Ottimistico in tutto il
    microservizio
14     @ExceptionHandler(ObjectOptimisticLockingFailureException.
    class)
15     public ResponseEntity<ErrorResponse>
    handleOptimisticLockingFailure(
16         ObjectOptimisticLockingFailureException ex) {
17
18         ErrorResponse error = new ErrorResponse(
19             HttpStatus.CONFLICT.value(),
20             "Conflict",
21             "Conflitto rilevato: la risorsa e' stata
    modificata da un'altra transazione concorrente."
22         );
23
24         return new ResponseEntity<>(error, HttpStatus.CONFLICT);
25     }
26
27     // Intercetta eventuali eccezioni generiche non gestite
28     @ExceptionHandler(Exception.class)
29     public ResponseEntity<ErrorResponse> handleGenericException(
    Exception ex) {
30         ErrorResponse error = new ErrorResponse(
31             HttpStatus.INTERNAL_SERVER_ERROR.value(),
32             "Internal Server Error",
33             "Si e' verificato un errore imprevisto nel server
    ."
34         );
35         return new ResponseEntity<>(error, HttpStatus.
    INTERNAL_SERVER_ERROR);
36     }
37 }

```

Listing 4.12: Classe GlobalExceptionHandler per l'intercettazione globale degli errori

Grazie a questo componente, l'infrastruttura di Spring agisce come un "filtro di

rete". Nel caso in cui il tentativo di scrittura su H2 fallisca per colpa della discrepanza nell'annotazione `@Version`, l'eccezione lanciata da Hibernate risale lo *stack trace* fino a essere catturata automaticamente da questo gestore. Il risultato è un blocco JSON compatto, predicibile e sicuro, che viene inoltrato all'orchestratore senza mai esporre dettagli interni o vulnerabilità del database.

## 4.3 Popolamento del Database in Memoria (Data Loader)

L'uso di H2 in modalità in-memory ha accelerato notevolmente lo sviluppo, ma mi ha posto di fronte a un limite intrinseco: a ogni riavvio dell'applicazione, i dati andavano persi. Per risolvere questo problema e disporre di un ambiente di test sempre pronto senza dover reinserire i dati manualmente a ogni avvio, ho dotato entrambi i microservizi di un componente di Data Loading.

Per farlo, ho sfruttato l'interfaccia `CommandLineRunner` di Spring Boot: ho creato una classe `@Component` che il framework esegue in automatico non appena il contesto applicativo è completamente avviato. Di seguito è riportato il `DataLoader` per il Servizio Utenti:

```
1 package it.vanvitelli.utenti.config;
2
3 import it.vanvitelli.utenti.model.Utente;
4 import it.vanvitelli.utenti.repository.UtenteRepository;
5 import org.springframework.beans.factory.annotation.Autowired;
6 import org.springframework.boot.CommandLineRunner;
7 import org.springframework.stereotype.Component;
8
9 @Component
10 public class DataLoader implements CommandLineRunner {
11
12     @Autowired
13     private UtenteRepository utenteRepository;
14
15     @Override
16     public void run(String... args) throws Exception {
17         // Pulisco il database per evitare duplicati
18         // in caso di riavvii a caldo
19         utenteRepository.deleteAll();
20
21         // Creazione di utenti di test
22         Utente u1 = new Utente(
```

```

23         "U123", "Raimondo", "Panico", true
24     );
25     Utente u2 = new Utente(
26         "U456", "Salvatore", "Giannone", false
27     );
28
29     // Salvataggio nel database H2
30     utenteRepository.save(u1);
31     utenteRepository.save(u2);
32
33     System.out.println(
34         "Database Utenti popolato con successo!"
35     );
36 }
37 }

```

Listing 4.13: Classe DataLoader.java (Servizio Utenti)

Ho replicato esattamente lo stesso approccio nel Servizio Colonnine, dove un **DataLoader** dedicato popola le stazioni di ricarica con stato iniziale **LIBERA**. Questa strategia si è rivelata molto pratica: automatizza completamente il setup dell'ambiente, garantisce la riproducibilità dei test e rende immediata la dimostrazione del sistema. A questo punto, il cuore transazionale della piattaforma è completo e pronto per essere testato.

## 4.4 Limiti Noti e Sviluppi Futuri

È importante segnalare alcuni limiti del prototipo attuale, frutto della scelta consapevole di mantenere ristretto lo scope dell'implementazione per focalizzarmi sulla concorrenza transazionale. Al momento, gli URL di comunicazione tra i servizi sono hardcoded (ad esempio, localhost:8082). Questa soluzione è valida per una demo locale, ma in un ambiente di produzione sarebbe necessario introdurre strumenti come il Service Discovery (es. Netflix Eureka) e un API Gateway per instradare le richieste in modo dinamico, specialmente in scenari di ridimensionamento orizzontale (scaling).

## 4.5 Conclusioni del Capitolo

Questo capitolo ha concretizzato le astrazioni progettuali in moduli software operativi. Spring Boot ha giocato un ruolo chiave, azzerando i tempi di configurazione e permettendomi di concentrarmi sulla logica di business e sulle interazioni di rete. Suddividere il dominio ha richiesto soluzioni ad hoc: dall'isolamento dei database in memoria, all'orchestrazione sincrona con `RestTemplate`, fino alla gestione della concorrenza tramite Lock Ottimistico. L'integrazione finale del Data Loading ha infine completato il setup automatico dell'ambiente di test. Una volta gettate queste solide fondamenta, il sistema è maturo per la validazione. Il prossimo capitolo sarà dedicato al collaudo delle API, dove l'infrastruttura verrà messa alla prova con test sotto carico e simulazioni di scenari di errore.

# Capitolo 5

## Collaudo e Validazione del Sistema

A differenza di un sistema monolitico, dove il collaudo si concentra prevalentemente sull'integrità del flusso di esecuzione interno, un'architettura a microservizi richiede un approccio strutturato su più livelli. Non è sufficiente testare la logica di business in isolamento: occorre validare anche le interazioni di rete, la corretta gestione delle transazioni concorrenti e la resilienza ai guasti.

### 5.1 Strategia e obiettivi del collaudo

Per validare il sistema in modo efficace, ho strutturato la strategia di collaudo in tre livelli incrementali:

1. **Unit Test (Test di Unità):** Ho isolato i componenti dal livello di persistenza tramite mock per testare la logica di business pura. Questa fase mi ha permesso di verificare le regole applicative (come il controllo dell'abilitazione) in modo deterministico, indipendentemente dal database.
2. **Integration Test (Test di Integrazione):** Ho verificato il comportamento del singolo microservizio nel suo complesso, coinvolgendo il reale database H2 in memoria. Questo livello è stato il più critico: ho simulato richieste HTTP concorrenti per testare l'intervento del Lock Ottimistico e la corretta risoluzione dei conflitti.
3. **Collaudo End-to-End (E2E) Cross-Servizio:** La prova finale del sistema. Ho avviato tutti i servizi simultaneamente e li ho interrogati con un client REST per dimostrare l'orchestrazione di rete, la serializzazione dei JSON e la coerenza del risultato finale per il Guidatore EV.

Nei paragrafi successivi verranno presentati nel dettaglio gli strumenti utilizzati per ciascuna fase, i frammenti di codice dei test più significativi e i risultati ottenuti.

## 5.2 Unit Test: Validazione della Logica di Business

Ho avviato la validazione logica sul Microservizio Utenti, con l'obiettivo di verificare la rigidità delle policy di autorizzazione prima che qualsiasi richiesta venisse inoltrata all'infrastruttura fisica.

Per garantire test rapidi e deterministici, ho scelto di non caricare l'intero Application Context di Spring Boot e di non istanziare il database H2. Ho invece utilizzato il framework Mockito per creare oggetti simulati delle dipendenze esterne del controller, ossia `UtenteRepository` e `RestTemplate`.

Il frammento di codice seguente (Listing 5.1) illustra lo scenario di test più critico per la sicurezza dell'anagrafica: il tentativo di sblocco da parte di un guidatore con abbonamento revocato o sospeso.

```
1 @Test
2 void testAvviaRicarica_UtenteNonAbilitato() {
3     // 1. Arrange: Configurazione del Mock
4     String matricola = "U999";
5     Utente utenteDisabilitato = new Utente(matricola, "Raimondo",
6     "Panico", false);
7
8     when(utenteRepository.findByMatricola(matricola))
9         .thenReturn(Optional.of(utenteDisabilitato));
10    Map<String, Object> payload = Map.of(
11        "idUtente", matricola,
12        "idColonnina", 1
13    );
14    // 2. Act: Invocazione diretta del metodo
15    ResponseEntity<?> response = ricaricaController.avviaRicarica
16    (payload);
17    // 3. Assert: Verifica del blocco autorizzativo
18    assertEquals(HttpStatus.FORBIDDEN, response.getStatusCode());
19    Map<String, String> body = (Map<String, String>) response.
20    getBody();
21    assertEquals("Abbonamento disabilitato", body.get("errore"));
```

Listing 5.1: Test di unità per la validazione dell'abbonamento tramite Mockito

Isolare il livello web con questa strategia mi ha dato la certezza che la logica decisionale sui crediti fosse intrinsecamente solida, rimandando le verifiche di rete e persistenza ai successivi test di integrazione.

## 5.3 Integration Test: Concorrenza e Optimistic Locking

Per il Servizio Colonnine ho adottato un approccio di testing differente, incentrato sull'interazione reale con il database e sulla gestione delle transazioni concorrenti. Il cardine di questo test è stato validare il comportamento del Lock Ottimistico in uno scenario di forte stress. Per simulare fedelmente un ambiente di produzione, ho utilizzato `@SpringBootTest` impostando la porta del web environment su un valore casuale (`RANDOM_PORT`), sollevando così l'intero contesto applicativo Spring, incluso il server Tomcat e il database H2. Per riprodurre la collisione, ho sfruttato il costrutto `CompletableFuture`, inviando simultaneamente due richieste HTTP POST di occupazione sulla stessa risorsa, simulando due utenti che agiscono nello stesso istante.

```
1 @Test
2 void testOccupazioneConcorrente_OptimisticLocking() throws
   Exception {
3     Map<String, String> payloadUtente1 = Map.of("idUtente", "U123
   ");
4     Map<String, String> payloadUtente2 = Map.of("idUtente", "U456
   ");
5     String url = "/api/colonnine/" + idColonninaTest + "/occupa";
6
7     // 1. Lancio di due richieste concorrenti su thread separati
8     CompletableFuture<ResponseEntity<Map>> request1 =
   CompletableFuture.supplyAsync(() ->
9         restTemplate.postForEntity(url, payloadUtente1, Map.
   class)
10    );
11
12    CompletableFuture<ResponseEntity<Map>> request2 =
   CompletableFuture.supplyAsync(() ->
13        restTemplate.postForEntity(url, payloadUtente2, Map.
   class)
14    );
15
16    // 2. Attesa della risoluzione simultanea dei thread
17    CompletableFuture.allOf(request1, request2).join();
```

```

18
19     ResponseEntity<Map> response1 = request1.get();
20     ResponseEntity<Map> response2 = request2.get();
21
22     // 3. Verifica che l'Optimistic Lock abbia gestito la
23     collisione (HTTP 200 vs HTTP 409)
24     boolean unoHaSuccesso = response1.getStatusCode() ==
25     HttpStatus.OK
26                             || response2.getStatusCode() ==
27     HttpStatus.OK;
28     boolean unoInConflitto = response1.getStatusCode() ==
29     HttpStatus.CONFLICT
30                             || response2.getStatusCode() ==
31     HttpStatus.CONFLICT;
32
33     assertTrue(unoHaSuccesso, "Una richiesta deve passare con 200
34     OK");
35     assertTrue(unoInConflitto, "La seconda richiesta deve fallire
36     con 409 CONFLICT");
37 }

```

Listing 5.2: Test di integrazione per la concorrenza e la risoluzione dei conflitti

L'esito positivo del test automatizzato ha dimostrato la resilienza dell'architettura alle race condition. Come mostra il codice, le asserzioni finali verificano semplicemente che una richiesta ottenga un 200 OK e l'altra un 409 CONFLICT, senza preoccuparsi dell'ordine esatto. **Si è scelto di non testare in modo deterministico l'ordine esatto di arrivo (ovvero quale specifico thread risulti vincente) in quanto, in uno scenario di asincronia spinta mediata dallo *scheduler* del sistema operativo e dallo stack di rete interno, l'ordine di elaborazione non è né prevedibile né rilevante ai fini dell'integrità del database.** Ciò che conta è esclusivamente l'intervento a monte del Lock Ottimistico per prevenire la doppia assegnazione.

## 5.4 Collaudo End-to-End (E2E) Cross-Servizio

Per concludere il ciclo di validazione, ho eseguito un collaudo End-to-End (E2E) per verificare la comunicazione inter-processo tra i microservizi. In uno scenario reale, i servizi devono interagire in modo trasparente scambiandosi payload JSON via rete. Ho quindi avviato entrambi i servizi sulle rispettive porte (8081 e 8082). Usando Postman, ho simulato l'applicazione mobile del guidatore, inviando una richiesta POST con matricola e ID colonnina all'endpoint del Servizio Utenti.

Come mostrato in Figura 5.1, il Servizio Utenti ha validato l'utente nel proprio database, ha inoltrato la richiesta al Servizio Colonnine tramite RestTemplate, e ha infine restituito un codice **200 OK**. Questo esito positivo ha dimostrato la corretta integrazione e la solidità dell'intera catena architetturale progettata.



Figura 5.1: Risposta HTTP 200 OK generata in fase di collaudo End-to-End su Postman.

Per validare ulteriormente la robustezza transazionale, ho eseguito una seconda richiesta consecutiva con lo stesso payload verso il medesimo endpoint. Poiché la risorsa (Colonnina 1) risultava ormai marcata come **IN\_USO** nel database H2, il sistema ha correttamente rilevato l'anomalia. Come mostra la Figura 5.2, l'infrastruttura ha rifiutato l'operazione restituendo un codice **409 CONFLICT**, dimostrando un'efficace gestione dei conflitti e l'impossibilità di sovrascrivere stati già occupati.

**REQUEST**  
**POST** http://localhost:8081/api/ricarica/avvia  
**REQUEST BODY**  

```
{
  "idUtente": "U123",
  "idColonnina": 1
}
```

**409** **Conflict**  
The request could not be completed due to a conflict with the current state of the resource.  
**RESPONSE BODY**  

```
{
  "errore": "La colonnina e' occupata o si e' verificato un conflitto"
}
```

**PARSED FIELDS**  

|        |                                                          |
|--------|----------------------------------------------------------|
| errore | La colonnina e' occupata o si e' verificato un conflitto |
|--------|----------------------------------------------------------|

**TEST RESULTS**  

**Status code is 200**  
expected response to have status code 200 but got 409

1 test run · 0 passed · 1 failed

Figura 5.2: Gestione del conflitto: risposta HTTP 409 ottenuta al tentativo di prenotare una colonnina già occupata.

## 5.5 Discussione dei risultati e limiti del collaudo

La suite di test eseguita ha validato il comportamento del sistema su diversi fronti. Gli unit test hanno accertato la robustezza della logica di business isolandola dalle dipendenze esterne, mentre i test di integrazione hanno confermato l'efficacia del Lock Ottimistico sul livello di persistenza. È doveroso, però, riconoscere alcuni limiti metodologici intrinseci a questo collaudo. Innanzitutto, il collaudo End-to-End (E2E) descritto è stato eseguito manualmente tramite Postman. Sebbene efficace per un'ispezione visiva, un simile approccio risulta metodologicamente fragile se confrontato con suite di test automatizzate (come Cucumber o RestAssured), che diventano obbligatorie in una pipeline di Continuous Integration aziendale. Inoltre, H2 è un database in memoria e non riproduce le dinamiche reali di un DBMS relazionale: in produzione subentrerebbero tempi di I/O su disco e problematiche legate ai colli di bottiglia del connection pool. Infine, eseguire i microservizi in locale su *localhost* annulla completamente la latenza di rete, celando potenziali problemi di timeout che, invece, costituiscono criticità tipiche delle infrastrutture cloud.

## 5.6 Conclusioni del capitolo

Questo capitolo conclude la fase pratica del progetto, confermando che l'infrastruttura implementata soddisfa i requisiti progettuali. Gli strumenti di testing offerti da Spring Boot hanno permesso di scrivere test automatizzati e replicabili, evitando la necessità di avvii manuali del server. Al contempo, la simulazione diretta con Postman ha dimostrato che i servizi comunicano e si orchestrano correttamente. I meccanismi di controllo della concorrenza hanno funzionato come atteso, bloccando le richieste simultanee e salvaguardando il database da stati inconsistenti. In definitiva, i risultati ottenuti validano la solidità del codice e lasciano il progetto in uno stato maturo, potenzialmente pronto per un inserimento in ambienti di staging o produzione.

# Capitolo 6

## Conclusioni e Sviluppi Futuri

### 6.1 Conclusioni

L'obiettivo principale di questa tesi era progettare e sviluppare un'infrastruttura backend per un hub di ricarica di veicoli elettrici basato su microservizi. Rispetto a un monolite tradizionale, la separazione dei domini utenti e colonnine ha garantito una maggiore flessibilità e ha permesso una manutenzione indipendente dei due moduli. Durante lo sviluppo, Spring Boot ha notevolmente semplificato la configurazione iniziale, consentendomi di concentrarmi sulla logica di business e sulle orchestrazioni di rete. Le chiamate REST si sono rivelate efficaci, ma hanno imposto un focus rigoroso sulla consistenza dei dati. La gestione della concorrenza ha rappresentato una delle sfide architetturali più complesse; il Lock Ottimistico ha risolto efficacemente il problema, prevenendo sovrascritture indesiderate senza sacrificare le prestazioni. Il collaudo finale ha validato le scelte progettuali: dagli unit test isolati ai collaudi End-to-End, il sistema ha dimostrato una gestione autonoma e robusta delle anomalie e dei conflitti. L'infrastruttura realizzata si configura, quindi, come una solida base rispondente ai requisiti di partenza e pronta per essere estesa con nuove funzionalità.

### 6.2 Sviluppi Futuri

Sebbene il sistema realizzato costituisca una base architetturale solida, l'infrastruttura attuale è stata pensata principalmente per un ambiente di sviluppo. Per un eventuale rilascio in produzione, ritengo che le seguenti evoluzioni tecnologiche sarebbero indispensabili. Il primo passo naturale sarebbe la containerizzazione dei microservizi tramite Docker. L'inserimento di ogni servizio in un proprio container garantisce l'isolamento completo degli ambienti. A questo si

potrebbe affiancare un orchestratore come Kubernetes, fondamentale per scalare dinamicamente le istanze in base al carico di rete. Per formalizzare questa evoluzione, ho elaborato il Diagramma di Deployment concettuale di Figura 6.1).

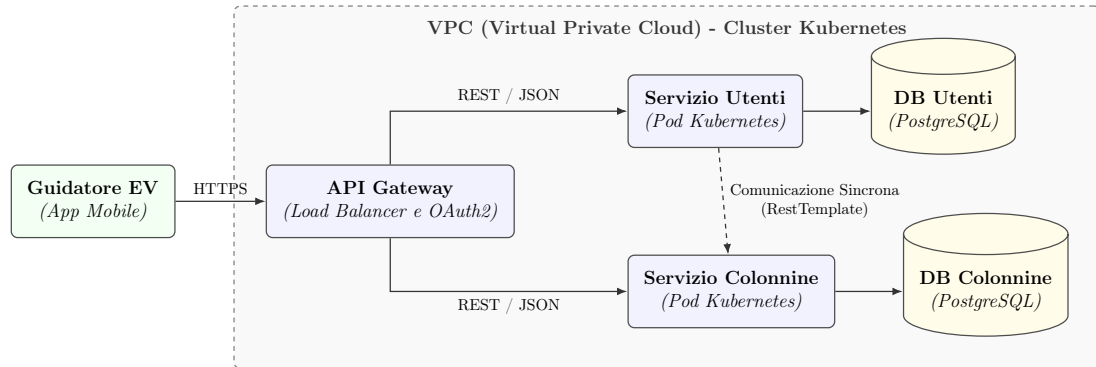


Figura 6.1: Diagramma di Deployment Concettuale: architettura a microservizi containerizzata con API Gateway e orchestrazione Kubernetes.

Come evidenziato nella topologia di rete, l'API Gateway gioca un ruolo centrale: funge da unico punto di ingresso (Single Point of Entry) per i client, mascherando gli indirizzi e le porte dei servizi sottostanti. Oltre a instradare il traffico, il Gateway permette di gestire centralmente l'autenticazione (ad esempio tramite JWT in OAuth2) e di bilanciare il carico. I singoli nodi Spring Boot e le relative istanze di database PostgreSQL, racchiusi all'interno del cluster Kubernetes, rimangono così confinati in una *Virtual Private Cloud* (VPC). Essendo irraggiungibili direttamente da Internet, il livello di sicurezza perimetrale risulta drasticamente innalzato, prevenendo attacchi diretti ai moduli interni.

### 6.2.1 Containerizzazione e Infrastruttura DevOps: Docker e Docker Compose

Per rendere concreta l'evoluzione verso il cloud ipotizzata nel diagramma di deployment, il sistema deve essere svincolato dalla macchina locale. La soluzione standard prevede la stesura di un Dockerfile Multi-stage per ciascun microservizio. L'approccio Multi-stage è essenziale per ottimizzare le risorse: un primo stadio (basato su Maven) compila il fat-JAR, mentre un secondo stadio, molto più leggero (basato su una JRE minima come Alpine Linux), si limita a eseguire l'applicativo. Questo riduce drasticamente le dimensioni dell'immagine finale.

```

1 # Fase 1: Build
2 FROM maven:3.9.4-eclipse-temurin-17 AS build
3 WORKDIR /app
4 COPY pom.xml .
5 COPY src ./src
6 RUN mvn clean package -DskipTests
7
8 # Fase 2: Run
9 FROM eclipse-temurin:17-jre-alpine
10 WORKDIR /app
11 COPY --from=build /app/target/servizio-colonnine-0.0.1-SNAPSHOT.
    jar app.jar
12 EXPOSE 8082
13 ENTRYPOINT ["java", "-jar", "app.jar"]

```

Listing 6.1: Esempio di Dockerfile Multi-stage per il microservizio

Oltre ai singoli container, l'orchestrazione locale dell'intero ecosistema (comprendente i due microservizi e, nell'ottica di abbandonare H2, due istanze di database PostgreSQL) viene definita tramite l'*Infrastructure as Code* (IaC). Di seguito è riportata la bozza del `docker-compose.yml` necessario per avviare la piattaforma con un singolo comando `docker compose up`:

```

1 version: '3.8'
2 services:
3   db-utenti:
4     image: postgres:15-alpine
5     environment:
6       POSTGRES_USER: admin
7       POSTGRES_PASSWORD: password
8       POSTGRES_DB: utenti_db
9     ports:
10      - "5432:5432"
11
12   servizio-utenti:
13     build: ./servizio-utenti
14     ports:
15      - "8081:8081"
16     environment:
17      - SPRING_DATASOURCE_URL=jdbc:postgresql://db-utenti:5432/
        utenti_db
18     depends_on:
19      - db-utenti
20
21   db-colonnine:
22     image: postgres:15-alpine

```

```

23     environment:
24         POSTGRES_USER: admin
25         POSTGRES_PASSWORD: password
26         POSTGRES_DB: colonnine_db
27     ports:
28     - "5433:5432"
29
30     servizio-colonnine:
31         build: ./servizio-colonnine
32         ports:
33         - "8082:8082"
34         environment:
35             - SPRING_DATASOURCE_URL=jdbc:postgresql://db-colonnine
36               :5432/colonnine_db
37         depends_on:
38             - db-colonnine

```

Listing 6.2: Definizione del docker-compose.yml per l'intero cluster

Questa configurazione mostra l'isolamento richiesto dai microservizi: ogni applicativo punta al proprio container database, eliminando il rischio di cross-query. Inoltre, l'uso di variabili d'ambiente per iniettare l'URL del database rispetta i principi della Twelve-Factor App, separando la configurazione dal codice e rendendo il rilascio indipendente dall'ambiente (Sviluppo, Staging, Produzione). Per quanto riguarda la persistenza, H2 andrebbe fisiologicamente sostituito da un DBMS relazionale robusto come PostgreSQL. Infine, per disaccoppiare ulteriormente i servizi, si potrebbe valutare la transizione a un'architettura Event-Driven, sostituendo le chiamate REST sincrone con un Message Broker (come Kafka o RabbitMQ). Sostituendo le chiamate REST sincrone con un Message Broker, il sistema diventerebbe più tollerante ai guasti, evitando che il disservizio temporaneo di un modulo si propaghi a cascata all'intera infrastruttura.

# Appendice: Configurazione Maven (pom.xml)

Per garantire la riproducibilità dell'ambiente e documentare le librerie impiegate, riporto di seguito il pom.xml integrale del Servizio Colonnine. L'uso della versione 3.3.0 di Spring Boot e l'inclusione mirata delle sole dipendenze essenziali (Web, JPA, H2 e Test) riflettono la scelta di mantenere il fat-JAR il più leggero e isolato possibile.

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="
  http://www.w3.org/2001/XMLSchema-instance"
3     xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
  https://maven.apache.org/xsd/maven-4.0.0.xsd">
4   <modelVersion>4.0.0</modelVersion>
5   <parent>
6     <groupId>org.springframework.boot</groupId>
7     <artifactId>spring-boot-starter-parent</artifactId>
8     <version>3.3.0</version>
9     <relativePath/>
10  </parent>
11  <groupId>com.example</groupId>
12  <artifactId>servizio-colonnine</artifactId>
13  <version>0.0.1-SNAPSHOT</version>
14  <name>servizio-colonnine</name>
15  <description>Progetto Tesi Colonnine</description>
16  <properties>
17    <java.version>17</java.version>
18  </properties>
19  <dependencies>
20    <!-- Database in memoria H2 -->
21    <dependency>
22      <groupId>com.h2database</groupId>
23      <artifactId>h2</artifactId>
24      <scope>runtime</scope>
25    </dependency>
26
27    <!-- JPA e Hibernate per mappare le tabelle (Optimistic
  Lock) -->
```

```

28     <dependency>
29         <groupId>org.springframework.boot</groupId>
30         <artifactId>spring-boot-starter-data-jpa</artifactId>
31     </dependency>
32
33     <!-- Modulo Web per i Controller REST -->
34     <dependency>
35         <groupId>org.springframework.boot</groupId>
36         <artifactId>spring-boot-starter-web</artifactId>
37     </dependency>
38
39     <!-- Modulo Ufficiale per i Test (include TestRestTemplate)
40     -->
41     <dependency>
42         <groupId>org.springframework.boot</groupId>
43         <artifactId>spring-boot-starter-test</artifactId>
44         <scope>test</scope>
45     </dependency>
46 </dependencies>
47
48 <build>
49     <plugins>
50         <plugin>
51             <groupId>org.springframework.boot</groupId>
52             <artifactId>spring-boot-maven-plugin</artifactId>
53             <version>3.3.0</version>
54         </plugin>
55     </plugins>
56 </build>
57 </project>

```

Listing 6.3: File pom.xml integrale del Servizio Colonnine

*Nota metodologica:* Il file `pom.xml` del Servizio Utenti è stato omesso per evitare ridondanza. Condividendo il medesimo stack (Java 17 e Spring Boot) e dipendenze pressoché identiche, la sua inclusione non avrebbe aggiunto valore. Ho pertanto scelto di concentrare l'attenzione sul modulo che gestisce il cuore transazionale dell'infrastruttura.

# Bibliografia e Sitografia

## Testi e Pubblicazioni Accademiche

Newman, S. (2021). *Building Microservices: Designing Fine-Grained Systems* (2nd ed.). O'Reilly Media.

Richardson, C. (2018). *Microservices Patterns: With examples in Java*. Manning Publications.

Fowler, M., & Lewis, J. (2014). *Microservices: a definition of this new architectural term*. MartinFowler.com.

Disponibile all'indirizzo:

<https://martinfowler.com/articles/microservices.html>

Gilbert, S., & Lynch, N. (2002). *Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services*. ACM SIGACT News, 33(2), 51-59.

Garcia-Molina, H., & Salem, K. (1987). *Sagas*. ACM SIGMOD Record, 16(3), 249-259.

Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures*. Tesi di Dottorato, University of California, Irvine.

## Documentazione Ufficiale e Sitografia

Spring Documentation. *Spring Boot Reference Guide*.

Disponibile all'indirizzo:

<https://docs.spring.io/spring-boot/index.html>

Hibernate ORM Documentation. *Optimistic Locking and Concurrency*.

Disponibile all'indirizzo: <https://hibernate.org/orm/documentation/>

H2 Database Engine. *H2 Documentation*.

Disponibile all'indirizzo: <https://h2database.com/html/main.html>

Apache Maven Project. *Maven Reference and Guides*.

Disponibile all'indirizzo: <https://maven.apache.org/guides/>

JUnit 5 User Guide. *Software testing framework for Java*.

Disponibile all'indirizzo:

<https://junit.org/junit5/docs/current/user-guide/>

Mockito Documentation. *Mockito framework site*.

Disponibile all'indirizzo: <https://site.mockito.org/>

Postman Learning Center. *API Testing and Validation*.

Disponibile all'indirizzo: <https://learning.postman.com/>

Open Charge Alliance. *OCPP (Open Charge Point Protocol) Core Specifications*.

Disponibile all'indirizzo: <https://www.openchargealliance.org/>

Docker Inc. *Docker Documentation: Multi-stage builds e Containerization*.

Disponibile all'indirizzo:

<https://docs.docker.com/build/building/multi-stage/>

Kubernetes Authors. *Kubernetes Components and Production Environment*.

Disponibile all'indirizzo:

<https://kubernetes.io/docs/concepts/overview/components/>

Swagger / SmartBear. *OpenAPI Specification 3.0*.

Disponibile all'indirizzo: <https://swagger.io/specification/>

# Ringraziamenti

Al mio Relatore, Prof. Antonio Esposito, per avermi guidato con pazienza, disponibilità e competenza durante la stesura di questo elaborato, fornendomi spunti ingegneristici di fondamentale importanza.

Alla mia famiglia, a cui devo tutto ciò che sono oggi. Per aver creduto in me fin dal primo giorno e per avermi sostenuto con sacrifici silenziosi e costanti, che non darò mai per scontati. Siete stati la mia roccia nei momenti di stanchezza, il mio porto sicuro dopo ogni difficoltà e la spinta decisiva per affrontare le sfide più dure. Mi avete insegnato il valore dell'impegno e della perseveranza, dandomi la serenità e i mezzi per inseguire le mie passioni. Questo traguardo non è solo una mia vittoria, ma anche vostra.

A tutti i miei amici e a coloro che mi hanno accompagnato e che ho avuto il piacere di conoscere durante questo percorso, rendendo questi anni semplicemente indimenticabili.

A Carlo, ormai per tutti "Gigi Buffon"

A Jacopo, il nostro personale "Pippo Inzaghi"

A Carmine, ribattezzato "Speedy Gonzales" per le sue corse inutili

A Nando, il nostro "Penaldo"

A Peppe, il muro della difesa

A Daniele, il nostro "Iniesta"

A Mariano, cittadino del mondo

A Sveva e Cristina, entrambe sempre di una gentilezza e disponibilità uniche.

A Lorenzo, la persona più buona che conosca, sempre pronto a dare una mano nei momenti di bisogno, nonché il nostro miglior tiratore di punizioni e lo chef numero uno al mondo per talento e pazienza, decisamente da tre stelle Michelin.

A Vincenzo, il "bomber" indiscusso di Giugliano, le cui innumerevoli conquiste lo rendono un fuoriclasse soprattutto fuori dal campo.

A Raffaele, il vero maestro dell'informatica e l'organizzatore instancabile del gruppo: non dimenticherò mai che sei stato la primissima persona che ho conosciuto all'università, rompendo il ghiaccio per chiederti se convenisse comprare un tablet solo per avere l'abbonamento a DAZN per le partite. Sei una persona d'oro e ti voglio tanto bene.

A Salvatore, che per me è molto più di un amico, è un secondo fratello: abbiamo iniziato questa avventura nello stesso momento e l'abbiamo conclusa insieme, sostenendoci e incoraggiandoci a vicenda esame dopo esame, condividendo gioie e frustrazioni. Sono felice che abbiamo concluso questo percorso insieme e spero che questa amicizia forte che si è creata duri con il passare degli anni. Ti voglio tanto bene.

A Rocky, il mio fedele compagno a quattro zampe. Per le innumerevoli ore passate a tenermi compagnia (e a dormire) vicino a me mentre studiavo, per le feste indescrivibili che mi facevi ogni volta che staccavo gli occhi dallo schermo, e per la tua capacità unica di spazzare via lo stress e le preoccupazioni con un semplice scodinzolio. Anche tu sei stato una parte preziosa, silenziosa e insostituibile di questo viaggio.

A Francesca, mia dolce metà. Ti ho incontrata proprio nel bel mezzo di questo percorso, e sei stata il mio vero colpo di fulmine: sei arrivata all'improvviso e hai stravolto e illuminato tutto. Fin dal primissimo momento, hai scelto di credere in me, infondendomi costantemente il coraggio, la forza d'animo e la motivazione necessari per non cedere mai e per spingermi a concludere questo capitolo nel migliore dei modi. Questo è stato un percorso intenso, a tratti estenuante, ma reso incredibilmente più leggero dalla tua presenza costante e dai bellissimi momenti vissuti insieme. Non posso dimenticare la nostra prima cena, proprio il giorno del mio compleanno, quando mi hai spiazzato prendendo un dolce di nascosto per farmi la sorpresa e farmi spegnere le candeline. E poi le serate passate a guardare le stelle insieme a Squille, fino ad arrivare all'emozione del nostro primo anno di fidanzamento, quel magico 22 agosto. Sei sempre la mia certezza più grande, una presenza fissa e rassicurante. Mi hai supportato con una dedizione incredibile e, soprattutto, mi hai sopportato nei periodi di maggiore stress e tensione, quando gli esami erano alle porte. Custodirò per sempre il ricordo delle innumerevoli volte in cui sei rimasta sveglia fino a tarda notte al mio fianco, sfidando la stanchezza solo per ascoltarmi ripetere pagine su pagine, prestandomi attenzione come se quegli argomenti fossero la cosa più importante del mondo. Sei stata e continui a essere la mia energia pura: quella scintilla inesauribile che mi porta a dare sempre il massimo. E nei momenti più bui, quando un esame non andava come previsto, eri sempre tu a rimettermi in piedi, a consolarmi con dolcezza e a ricordarmi di non arrendermi. Se oggi riesco a tagliare questo traguardo, una parte immensa del merito è tua. Grazie di tutto, e grazie per esserci sempre. Ti amo.